



UT6. Gestión de la configuración. Refactorización y documentación

ENTORNOS DE DESARROLLO

Análisis de código

Análisis de código

Existen en el mercado herramientas de análisis de código que nos ayudan a identificar problemas derivados de la codificación de los programas.

El análisis de código estático se define como una opción de prueba para el software de las aplicaciones o programas que usa métodos de revisión que detectan los potenciales fallos, como errores de funcionamiento, bugs o riesgos en el sistema.

Además de esto, el análisis de código estático ofrece la posibilidad de identificar los errores antes de que sucedan en el sistema, lo que contribuye a la reducción de costes ocasionados por la detección y consecuente corrección tardía de las dificultades.

Cabe resaltar que este tipo de análisis estático destaca gracias a que no requiere de la ejecución del código para poder realizar sus funciones.

Características del análisis de código

Dentro de las características del proceso de análisis estático de código podemos destacar que permite que los equipos de desarrollo de software incluyan una capa adicional de control de calidad automatizado, lo que hace más fácil la identificación de problemas de detección complicada. Para ello del análisis estático utilizan procesos como las pruebas o test unitarios.

Este tipo de procedimientos se enfocan en la verificación del funcionamiento de los componentes de software en diversos entornos, a través del escáner del código fuente, en búsqueda de errores como fugas de memoria o incumplimiento de parámetros de codificación, entre otros.

SonarLint

SonarLint es una extensión gratuita de análisis estático de código que ayuda a los desarrolladores a mejorar la calidad del código directamente desde el entorno de desarrollo integrado (IDE).

Sonarlint permite usar un servidor sonar como SonarQube o utilizar reglas locales.

Actividad 1.

Instala el plugin Sonarlint en IntelliJ y utilízalo para verificar código. Busca información sobre como configurar Sonarlint.

Software de Gestión de la Configuración

Software de Gestión de la Configuración

El desarrollo de software ha cambiado considerablemente a lo largo de los años. Los proyectos han evolucionado desde pequeños programas desarrollados por equipos pequeños y centralizados, a aplicaciones enormemente complejas y continuamente actualizadas desarrolladas por equipos numerosos de programadores, ubicados en diferentes sitios y trabajando sobre entornos diferentes; además, a las empresas que las desarrollan se les exigen cada vez más amplias certificaciones de calidad de sus procesos.

Para gestionar esta compleja situación surgió lo que se denomina el Software de Gestión de la Configuración (SCM), que debe afrontar tres grandes dificultades:

- El volumen de las aplicaciones hace **difícil** su **mantenimiento**: durante el mantenimiento del software, se requiere en un plan de administración de la configuración y herramientas para dicha administración, de manera que se pueda seguir la pista y controlar las distintas versiones de los productos de trabajo que constituyen el producto de software. El rastreo y control de las múltiples versiones de un producto software son un aspecto significativo en el mantenimiento del software.

Software de Gestión de la Configuración

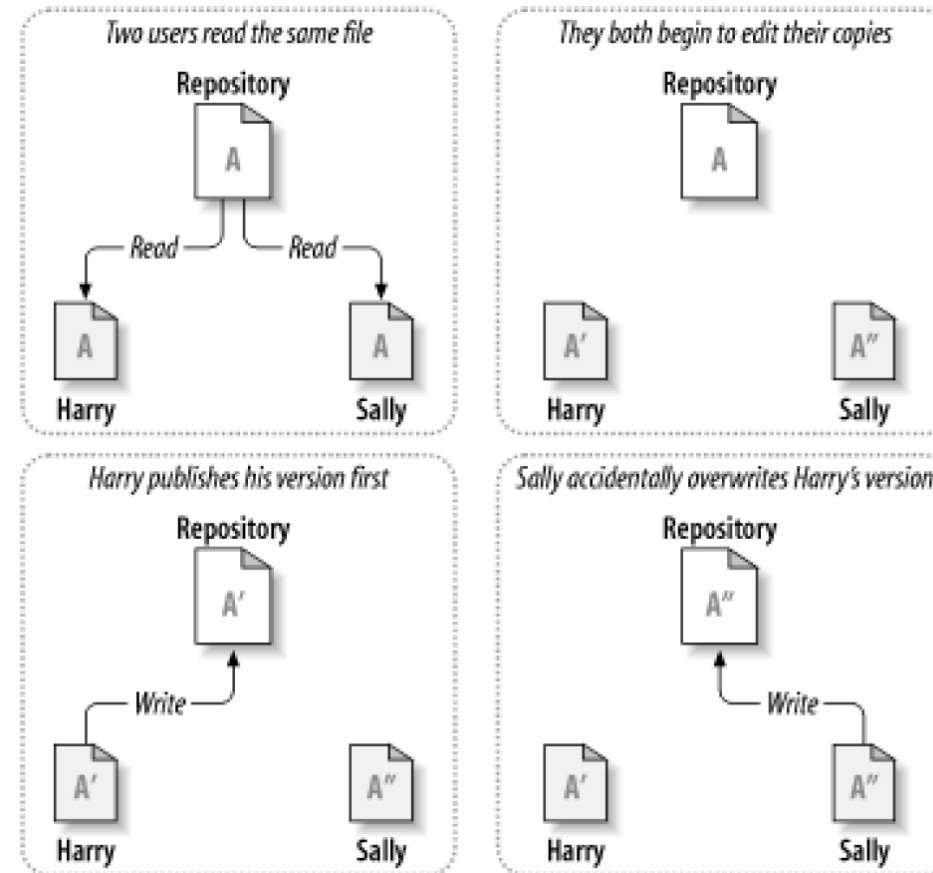
- El volumen de las aplicaciones, junto con el considerable número de programadores trabajando sobre ellas, hace que cobre enorme importancia la gestión de la privacidad.
- El tamaño de los grupos de programadores hace necesario facilitar la colaboración entre ellos y evitar que se produzcan interferencias destructivas.

Las herramientas de software para apoyar la administración de la configuración incluyen:

- Control de la configuración: proporciona información sobre la composición del producto, el número de versión y de revisión en curso, el estado en curso, la historia de las solicitudes de cambios para cada versión, cómo difieren las diferentes versiones y revisiones del producto, qué configuración de hardware requiere cada una de ellas, cuántas versiones se ven afectadas por un error específico, cuántos errores se corrigieron en cada versión y revisión, etc.
- Control de versiones: controla los distintos archivos que constituyen las diferentes versiones de un producto software; entre ellas pueden estar los ficheros fuente, los ficheros de datos, los documentos de apoyo,... De ellos se almacenará la historia detallada.

Software de Gestión de la Configuración

El escenario a evitar es el siguiente:



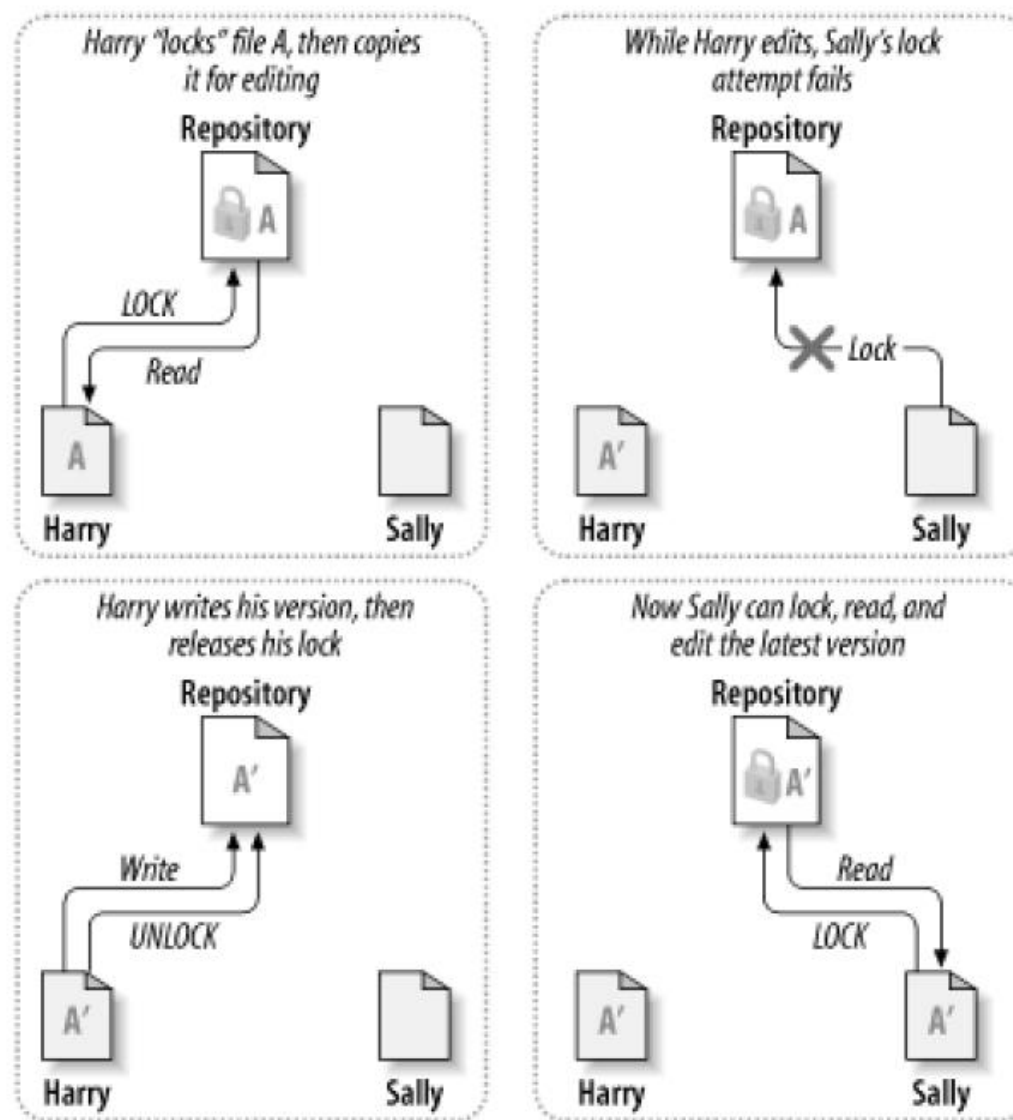
Modo exclusivo

Existen dos aproximaciones para impedir que suceda esa situación:

- Modo exclusivo (“lock-modify-unlock”): En este esquema para poder realizar cambios a un fichero es necesario comunicar al sistema qué fichero se desea modificar y el sistema se encargará de impedir que otro usuario pueda modificarlo simultáneamente (deberá esperar a que termine el primero). Una vez hecha la modificación, ésta se comparte con el resto de los colaboradores. Si se ha terminado de modificar un elemento entonces el sistema libera ese elemento para que otros lo puedan modificar. Este modo de funcionamiento es el que usa por ejemplo SourceSafe. En cambio, Subversion, aunque por defecto no usa este modo, dispone de mecanismos que permiten implementarlo.

El modelo lock-modify-unlock puede dar lugar a retardos innecesarios, ya sea porque algún usuario se puede olvidar de desbloquear un fichero sobre el que ya no está trabajando, o porque se hace imposible que otro usuario pueda estar trabajando simultáneamente sobre el mismo fichero, aunque vaya a modificar una zona diferente del archivo.

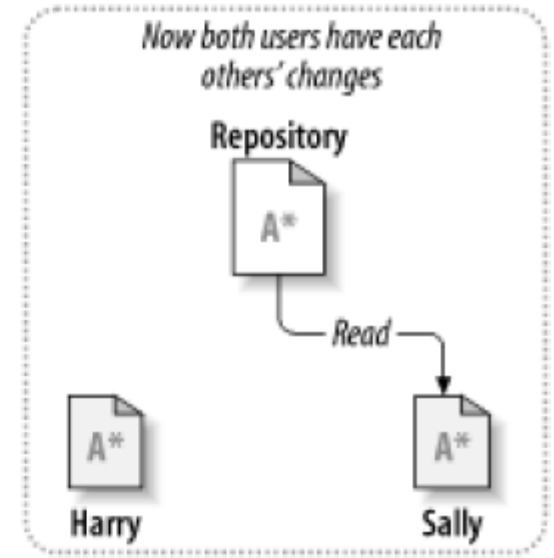
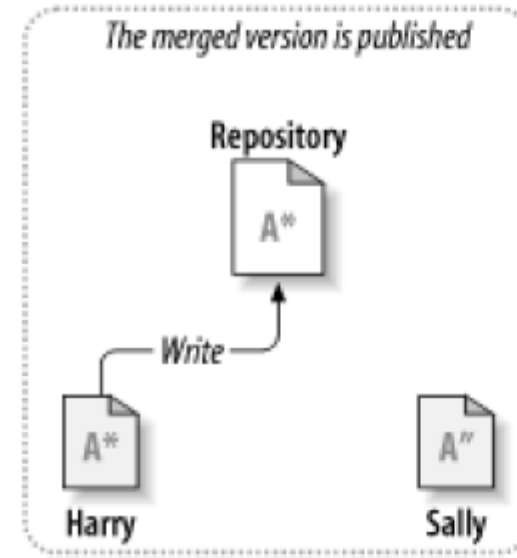
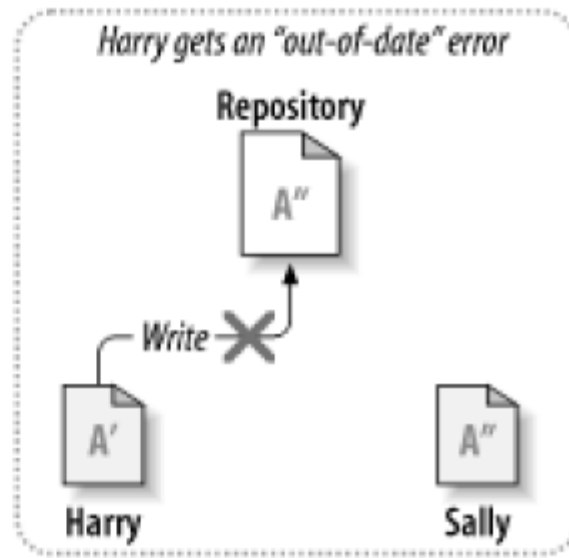
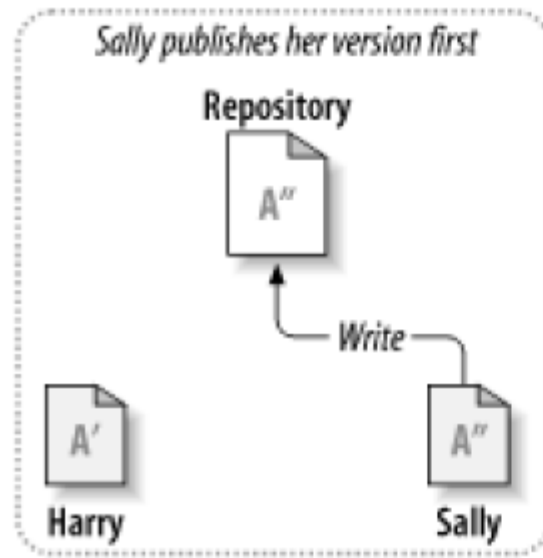
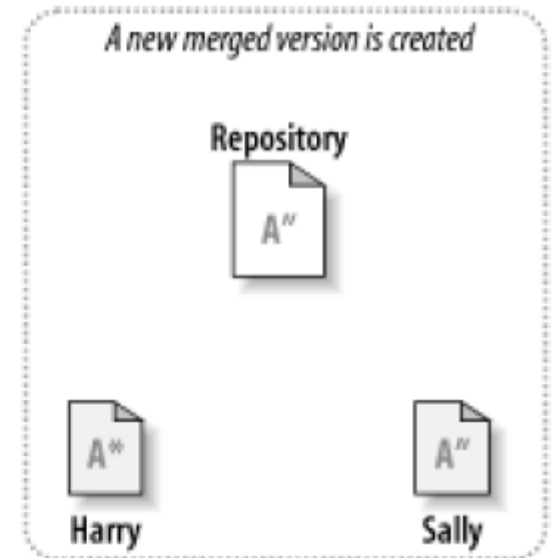
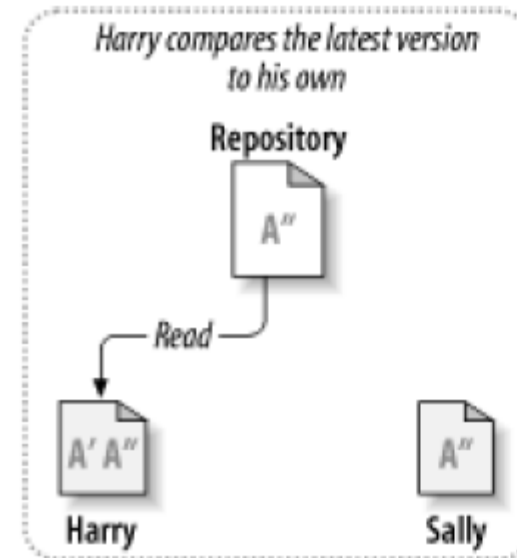
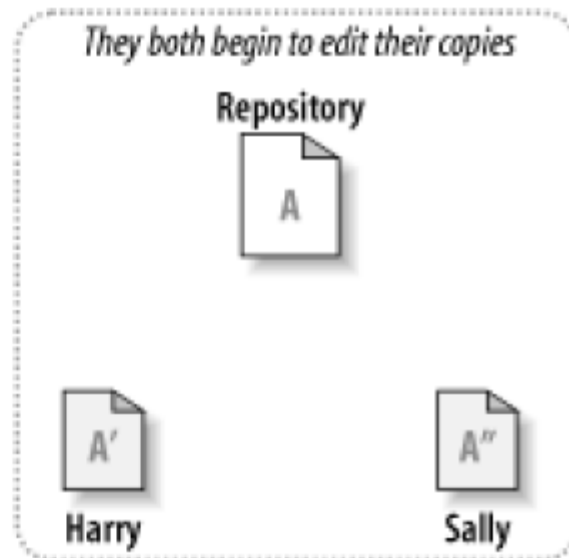
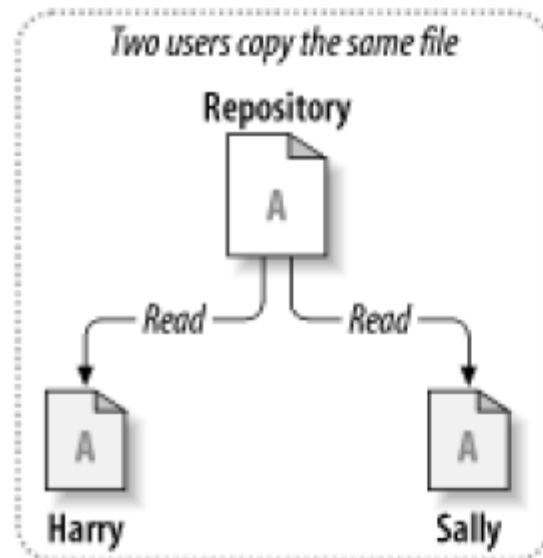
Modo exclusivo



Modo colaborativo

- Modo colaborativo (“copy-modify-merge”): En este esquema cada usuario modifica su copia local del fichero a cambiar y cuando el usuario decide compartir los cambios el sistema automáticamente intenta combinar las diversas modificaciones habidas; se mejora así la productividad. El principal problema es la posible aparición de conflictos que deban ser solucionados manualmente o las posibles inconsistencias que surjan al modificar el mismo fichero varias personas no coordinadas. Además, esta semántica no es apropiada para ficheros binarios. Los SCM Subversion o Git utilizan habitualmente este modo de funcionamiento.

Modo colaborativo



Modo colaborativo

El modelo copy-modify-merge puede parecer un poco caótico, pero en la práctica funciona muy bien. Los usuarios pueden trabajar en paralelo, sin esperarse unos a otros. Cuando trabajan en los mismos ficheros, la mayoría de sus cambios concurrentes no se superponen en absoluto; los conflictos no son frecuentes; y la cantidad de tiempo que se necesita para resolver los conflictos es mucho menor que el tiempo perdido por un sistema de bloqueo.

El modelo copy-modify-merge se basa en el supuesto de que los archivos son contextualmente fusionables; es decir, que la mayoría de los ficheros en el repositorio son archivos de texto (como el código fuente del programa). Pero para los archivos con formatos binarios (como los ficheros de imágenes, de sonido, hojas de cálculo, etc.), es imposible fusionar los cambios conflictivos. En estas situaciones es realmente necesario que los usuarios sigan turnos estrictos al cambiar el archivo, y por tanto es de aplicación el modelo lock-modify-unlock.

La arquitectura del SCM

Concepto de repositorio

Un sistema de control de versiones es un sistema que almacena las diferentes revisiones de los ficheros que se generan durante el desarrollo de una aplicación. De hecho, todo se guarda en el repositorio.

La descripción del repositorio se parece a la de un servidor de ficheros típico. Y, de hecho, el repositorio es una especie de servidor de archivos, pero no es el habitual. Lo que hace al repositorio especial es que **recuerda todos los cambios que se han escrito en él**: cada cambio a cada archivo, e incluso los cambios en el árbol de directorios en sí, tales como la adición, borrado y reubicación de archivos y directorios.

Cuando un cliente lee datos del repositorio, normalmente ve la última versión del árbol de ficheros. Pero el cliente también tiene la capacidad de ver estados previos del sistema de archivos. Por ejemplo, un cliente puede hacer preguntas históricas, como “¿Qué contenía este directorio el pasado miércoles?” o “¿Quién fue la última persona que cambió este fichero, y qué cambios hizo?”. Estas son el tipo de preguntas que precisan de un sistema de control de versiones para ser respondidas: sistemas que están diseñados para guardar y registrar los cambios en los datos con el tiempo.

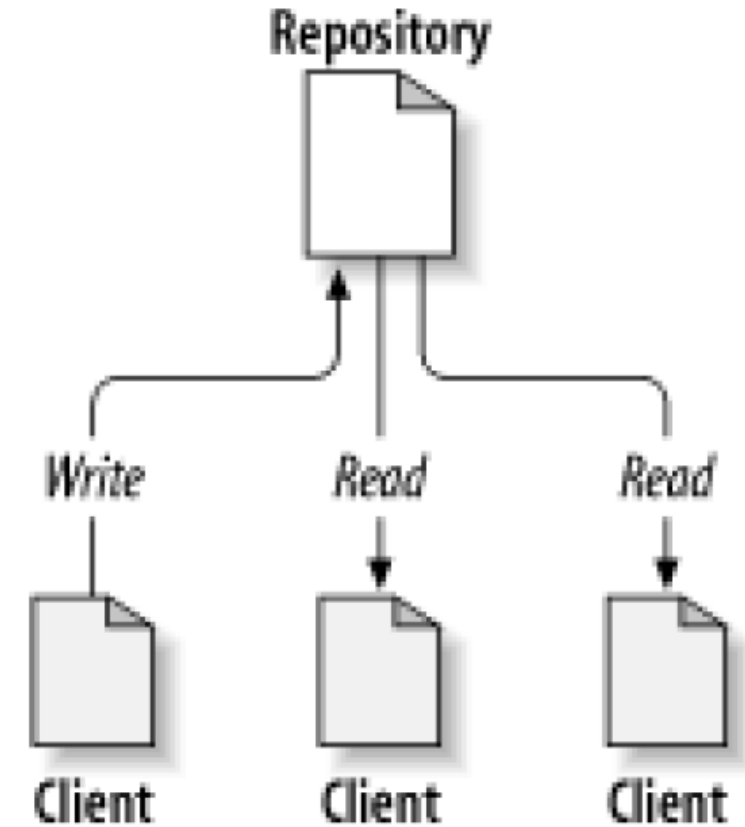
Concepto de repositorio

En casi todos los sistemas de control de versiones, **el repositorio es un lugar central que guarda la copia maestra de todas las versiones de los archivos del proyecto.** Algunos sistemas utilizan una base de datos como repositorio, otros utilizan archivos regulares, y algunos utilizan una combinación de los dos. De cualquier manera, el repositorio es claramente un componente fundamental del sistema. Por tanto, deberá estar ubicado en una máquina segura y fiable, y por supuesto debería ser objeto de copias de seguridad con regularidad.

Actualmente la mayoría de los sistemas de control de versiones soportan el funcionamiento en red; como desarrollador se puede acceder al repositorio sobre la red, con el repositorio actuando como servidor y las herramientas de control de versiones actuando como clientes. No importa donde estén los desarrolladores; siempre y cuando que puedan conectarse a través de una red al repositorio, podrán acceder de forma segura a todo el código de los proyectos y a su historia.

Concepto de repositorio

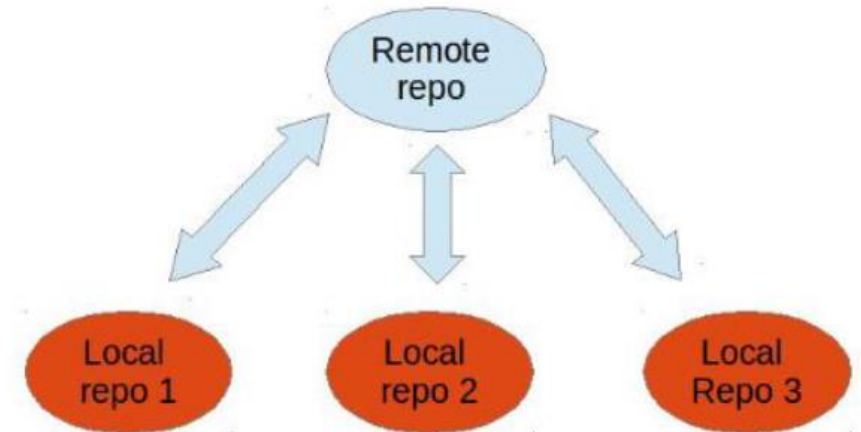
Cualquier número de clientes se puede conectar al repositorio, y leer o escribir archivos. Al escribir datos, un cliente hace que la información esté disponible a los demás; mediante la lectura de los datos, un cliente recibe información de otros.



Sistemas de control de versiones distribuidos

Algunos sistemas de control de versiones soportan la noción de múltiples repositorios: en lugar de un único repositorio central, cada desarrollador tiene su propio repositorio que tiene toda la historia del proyecto. Hacer un cambio en un fichero no implica la conexión a un repositorio remoto, el cambio se registra en el repositorio local. No obstante, para mantener la sincronía, cada desarrollador sigue enviando sus cambios al repositorio principal del proyecto.

Estos son sistemas de control de versiones distribuidos (DVCS); son muy utilizados cuando un gran número de desarrolladores necesitan operar semi-autónomamente. El principal ejemplo de ellos es Git, que se diseñó para gestionar el desarrollo del kernel de Linux y actualmente es utilizado también por muchos otros proyectos de código abierto, como Android o Eclipse, y por muchas organizaciones comerciales.

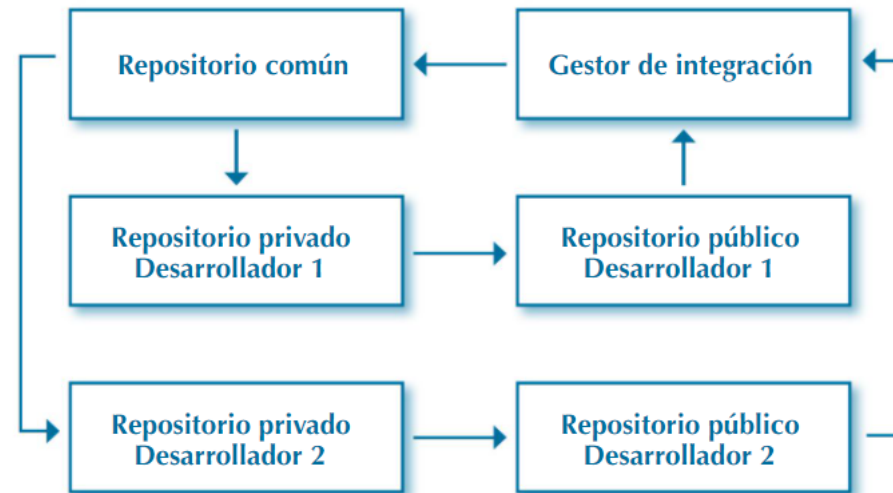


Sistemas de control de versiones distribuidos

En un DVCS el proceso de copia de un repositorio existente se denomina clonación. Normalmente hay un servidor central que mantiene un repositorio, y cada repositorio clonado es una copia completa de este repositorio. Cada clon contiene el historial completo de la colección de archivos y por tanto un repositorio clonado tiene la misma funcionalidad que el repositorio original. Cada clon del repositorio central puede intercambiar versiones de los archivos con otros clones, si bien esto normalmente lo realiza a través del repositorio del servidor central.

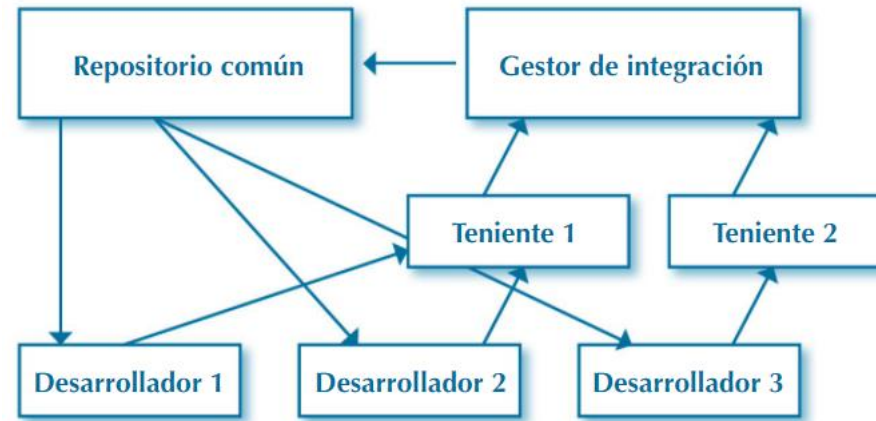
Flujo de trabajo centralizado

Es un sistema muy propio de la vieja escuela. Suele utilizarse un servidor centralizado y los desarrolladores van publicando sus actualizaciones de código en él. Estos sistemas funcionan de forma exclusiva, lo que implica que, si alguien está modificando un elemento ningún otro desarrollador, puede realizar actualizaciones de este



Flujo de trabajo con gestor de integración

En ocasiones, para un mayor control sobre las versiones del proyecto, se utiliza un gestor de integración, que suele ser una persona del equipo de desarrollo que actualiza los trabajos en el repositorio común. Algo parecido a un guardia de tráfico.



Los desarrolladores actualizan sus repositorios públicos y es el gestor de integración el encargado de actualizar el repositorio común con los trabajos públicos independientes de los desarrolladores. Este sistema suele ser muy utilizado en SCV distribuidos.

Contenido del repositorio

¿Qué ficheros de un proyecto se deben almacenar en el repositorio? Todo lo que sea esencial en el proyecto para que se pueda modificar, mejorar y crear nuevas versiones del mismo:

- Lo primero y más evidente es el **código fuente**. Sin él no se pueden corregir errores o implementar nuevas características.
- La mayoría de los proyectos tienen algún tipo de **archivos de construcción**. Entre los más comunes están los makefiles, los pom.xml de Maven y los build.xml de Ant. Estos necesitan ser almacenados para que se pueda compilar el código fuente.
- Otros elementos comúnmente almacenados en el repositorio son archivos de configuración de muestra, documentación, imágenes que se utilizan en la aplicación, y por supuesto las pruebas unitarias.

La determinación de lo que se debe incluir es fácil; habría que preguntarse: “Si yo no tuviera X, ¿podría trabajar en este proyecto?”; si la respuesta es no, debería ser incluido. Esta regla tiene una excepción: no se aplica a las herramientas que se deben utilizar; se debe incluir el archivo build.xml de Ant, pero no la totalidad del programa Ant.

Organización del repositorio

En el nivel más bajo, la mayoría de los sistemas de control de versiones tratan ficheros individuales. Cada archivo en el proyecto se almacena por su nombre en el repositorio. Sin embargo, esto es bastante bajo nivel. Un proyecto típico puede tener cientos o miles de archivos; por esta razón los sistemas de control de versiones permiten estructurar el repositorio, organizándolo en directorios. Por otra parte, una empresa puede tener docenas de proyectos. En Git, hay diferentes maneras de tratar con esta situación:

- Crear un repositorio y almacenar los diferentes proyectos en él. Para ello se crearía un directorio diferente en la raíz del repositorio para cada uno de los proyectos. Esta solución es adecuada con proyectos que necesitan tener una historia común porque cada uno de ellos es un componente diferente de una única aplicación, y, por tanto, se liberan versiones de todos ellos simultáneamente.
- Crear un repositorio diferente para cada proyecto. Esto permite que cada proyecto tenga una historia independiente, y la generación de sus versiones también lo será.
- Almacenar un repositorio dentro de otro repositorio, pero manteniendo las dos historias independientes. Esto se consigue utilizando submódulos. Esta solución permite establecer la dependencia de un proyecto respecto de un componente de terceros, o gestionar un proyecto interno que ha sido dividido en múltiples proyectos.

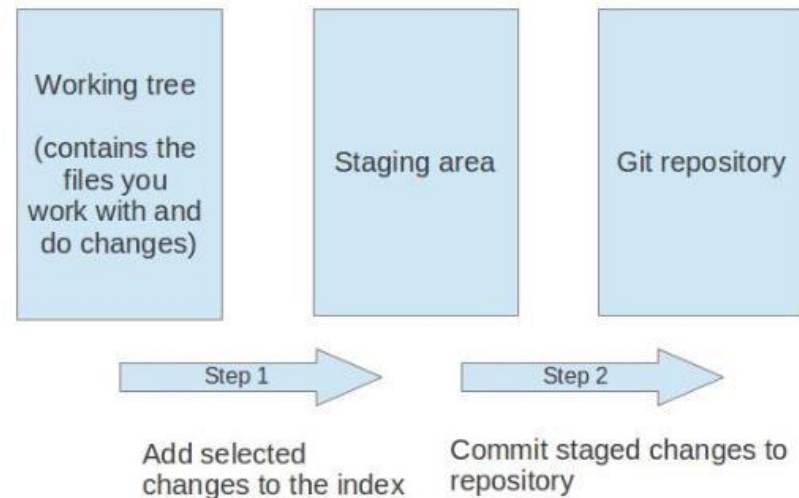
Árboles de trabajo y manipulación de archivos

Ya hemos comentado que el repositorio almacena los ficheros con toda su historia, pero ¿dónde se trabaja con esos ficheros? En Git la respuesta es “en el árbol de trabajo (working tree)”. El árbol de trabajo es la vista actual que el desarrollador tiene del conjunto de directorios y ficheros del repositorio. Toda la actividad del desarrollador se realiza sobre ellos.

Hay dos maneras de obtener un árbol de trabajo: comenzando un nuevo proyecto y pidiéndole a Git que cree un repositorio para él, o clonando un repositorio ya existente; en este segundo caso, Git genera un nuevo repositorio y crea un árbol de trabajo sincronizado con su línea principal de desarrollo (su rama maestra).

Árboles de trabajo y manipulación de archivos

El seguimiento de los cambios a los ficheros a lo largo del tiempo es la razón de ser del control de versiones. Si se modifican los ficheros del árbol de trabajo, por ejemplo, al crear un nuevo archivo o al cambiar un archivo existente, se deben realizar dos pasos en Git para registrar los cambios en el repositorio: primero agregar (add) los archivos seleccionados al área de preparación (staging area o index) y luego confirmar (commit) los cambios del área de preparación en el repositorio de Git.



Árboles de trabajo y manipulación de archivos

Por tanto, para que los cambios en el árbol de trabajo sean relevantes para Git, el primer paso será añadirlos al staging area, lo que almacenará en ésta una instantánea de esos archivos cambiados. El contar con un área de preparación del commit, permite ir modificando de manera incremental los archivos del árbol de trabajo hasta que se esté satisfecho con todos los cambios.

Una vez agregados los archivos seleccionados al área de preparación, de una vez se pueden confirmar (commit) todos los cambios para agregarlos permanentemente al repositorio de Git. La confirmación crea un nuevo objeto commit, que es una instantánea inmutable y persistente del staging area en el repositorio, que ha de incluir siempre un mensaje explicativo (log message) de los cambios. Por tanto, cada commit es un nuevo evento en la historia del repositorio.

Árboles de trabajo y manipulación de archivos

Un fichero en el árbol de trabajo de un repositorio Git puede tener diferentes estados. Estos estados son los siguientes:

- Sin seguimiento (untracked): el repositorio de Git no rastrea el archivo. Esto significa que el archivo nunca se ha incluido en ningún commit, ni tampoco está en la staging area.
- Con seguimiento (tracked): el fichero está comprometido y no tiene cambios en la staging area ni en el árbol de trabajo.
- Preparado (staged): El fichero ha sido incluido en la staging area, para ser incluido en el próximo commit.
- Modificado (modified): el archivo ha cambiado en el árbol de trabajo, pero el cambio no se ha introducido en la staging area.

Árboles de trabajo y manipulación de archivos

Un DVCS, tal como Git, requiere que el desarrollador comparta sus cambios con otros desarrolladores. Esto se consigue **pushing** (promocionando) los cambios a un repositorio compartido. Por otra parte, el desarrollador también deberá ser capaz de obtener las actualizaciones realizadas por los demás: para ello deberá traer (**fetching**) esos cambios del repositorio compartido y fusionarlos (**merging**) en su repositorio local, proceso denominado conjuntamente como **pulling** (extracción).

En ocasiones, dos desarrolladores, Harry y Sally, habrán clonado el mismo repositorio central, y estarán trabajando sobre el mismo archivo de sus respectivos repositorios locales. Sally enviará (push) sus cambios al repositorio compartido, y a continuación Harry intentará hacer lo mismo. El intento de Harry será rechazado, porque Git detecta que ha habido cambios en el repositorio central desde que Harry obtuvo su copia. Harry tendrá que obtener (fetch) esos cambios, fusionarlos (merge) con su repositorio local resolviendo los conflictos que surjan, y luego enviar (push) de nuevo sus cambios al repositorio compartido.

El control de versiones

Evolución de los ficheros

El repositorio de un sistema de control de versiones no se limita a almacenar la copia actual de cada uno de los archivos bajo su cuidado; almacena todas las versiones que se hayan comprometido.

La mayoría de los sistemas de control de versiones (Subversion, por ejemplo) realizan un seguimiento de la evolución de cada fichero. Para Git, en cambio, los nombres de los ficheros no son más que metadatos asociados a los contenidos para poder organizarlos; así pues, Git realiza un seguimiento del contenido (de las distintas líneas que constituyen los diferentes ficheros). Esto técnicamente tiene algunas ventajas, como reducir el espacio necesario para almacenar la historia del repositorio y permite detectar fragmentos de código fuente que se han movido o copiado de unos ficheros a otros.

En Git, cuando se comprometen (commit) los cambios, se crea un nuevo objeto commit en el repositorio. Este objeto identifica de forma exclusiva la nueva revisión del contenido del repositorio. Esta revisión se podrá recuperar más adelante, por ejemplo, si se desea ver el código fuente que la formaba. Desde cada commit, siempre se puede acceder a sus predecesores, retrocediendo por la historia del repositorio, pues cada commit conoce a sus antecesores inmediatos.

Ramificación y fusión de variantes

La historia de un proyecto puede avanzar simultáneamente a lo largo de varios caminos diferentes. Aquí es donde entran en juego las ramas (branches).

Una rama nace en el punto en el que la historia de los ficheros del repositorio diverge. Cada rama sigue la pista de los cambios que se han hecho en ella separadamente de las otras ramas, y por tanto tiene su propia historia. El usuario podrá seleccionar (checkout) sobre qué rama está trabajando en cada momento; esta rama es la rama activa.

Desde un punto de vista técnico, en Git una rama es simplemente un puntero con nombre al último commit de la rama. Cuando se está trabajando sobre una rama, si se crea un nuevo commit, el puntero de la rama avanza para apuntar al nuevo commit.

Por otra parte, cuando se está trabajando sobre una rama, HEAD es una referencia simbólica que apuntará a ésta. Si el desarrollador cambia de rama activa, el puntero HEAD referenciará al puntero de la nueva rama. Por tanto, HEAD suele apuntar a la rama activa; no obstante, HEAD puede apuntar directamente a un objeto commit, lo que se denomina modo detached-HEAD; en ese estado, la creación de un nuevo commit no hace avanzar ninguna rama.

Ramificación y fusión de variantes

La principal línea de desarrollo de un proyecto en Git se llama rama maestro (master). A partir de ella se pueden generar otras ramas. Una rama puede existir durante el resto del proyecto o sólo durante unas pocas horas.

Por ejemplo, si se desea desarrollar una nueva funcionalidad, o corregir algún bug, se puede crear una rama y hacer los cambios en esta rama sin afectar el estado de los archivos de la rama maestra. Una vez que la funcionalidad se haya terminado, sus cambios se fusionarán sobre la rama maestro.

En Git las ramas pueden ser creadas localmente sin tener que compartirlas con los demás desarrolladores. Esto tiene sus ventajas, pues permite, por ejemplo, crear ramas locales para experimentar, y compartir los resultados únicamente si se ha llegado a conclusiones provechosas.

Ramificación y fusión de variantes

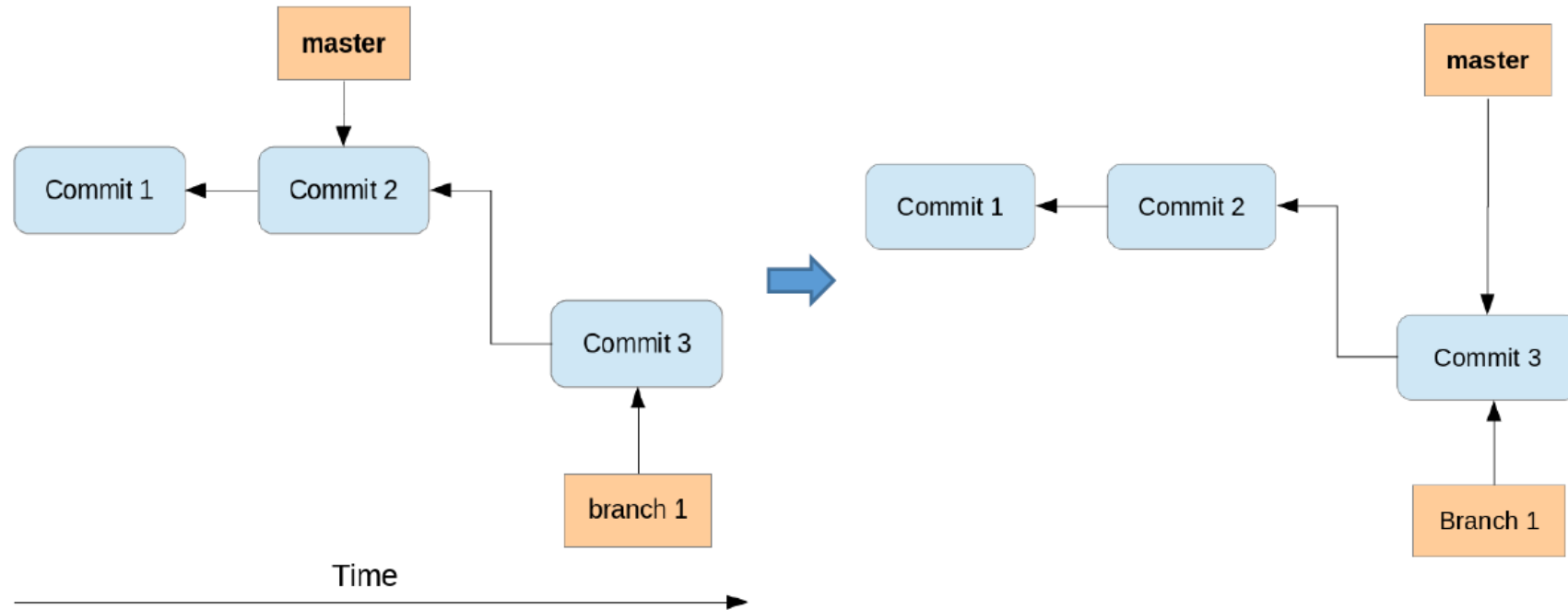
En ocasiones es necesario que dos ramas vuelvan a unirse. Es lo que se llama fusionado (merge). Fusionar una rama con otra consiste en combinar la historia de cambios de la primera rama con la de la segunda, de manera que ésta incorpore los cambios habidos en aquella.

Cuando el mismo fichero se modifica en dos ramas, al fusionarlas se pueden dar dos situaciones: que los dos cambios se hayan realizado en distintas zonas del fichero, en cuyo caso Git puede combinarlos automáticamente, o que los cambios hayan afectado a la misma zona del fichero, con lo que Git marcará esos cambios como conflictivos y esperará a que el desarrollador los combine manualmente.

Fusiones de variantes

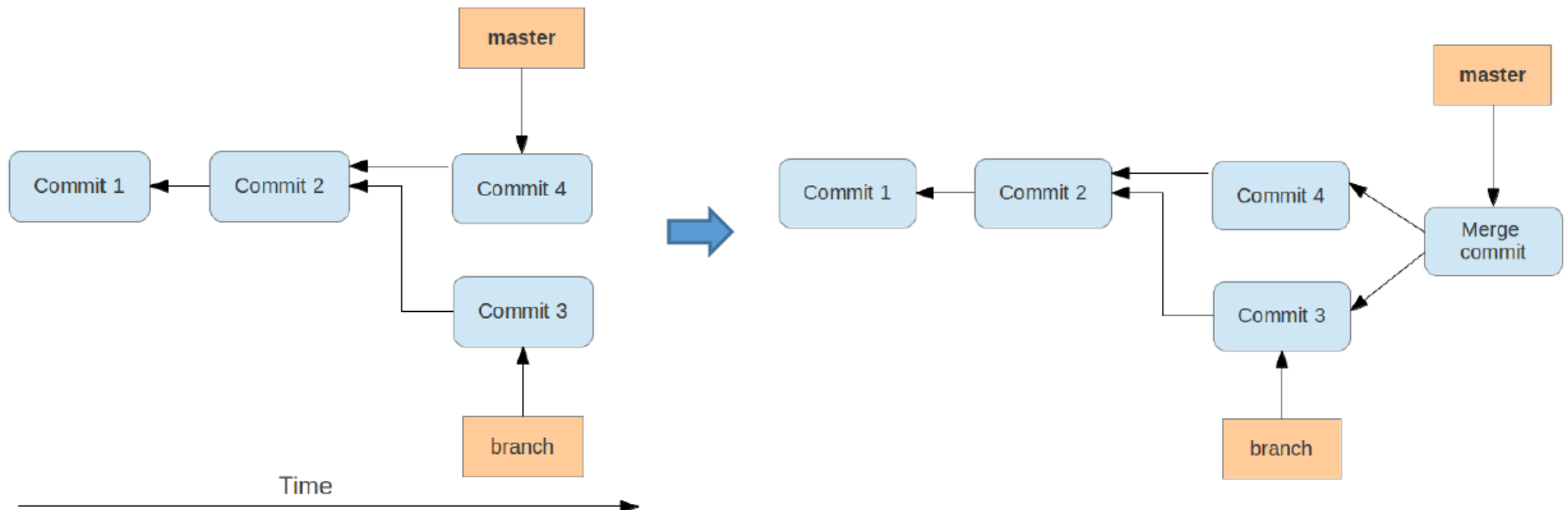
Git permite fusionar los cambios entre dos ramas. Para ello hay varias posibilidades:

- **Fast-forward merge:** Si los commits que se fusionan son sucesores directos del puntero HEAD de la rama actual, Git realiza una fusión de avance rápido (fast-forward merge). Ésta sólo mueve el puntero HEAD de la rama actual a la punta de la rama que se está fusionando.



Fusiones de variantes

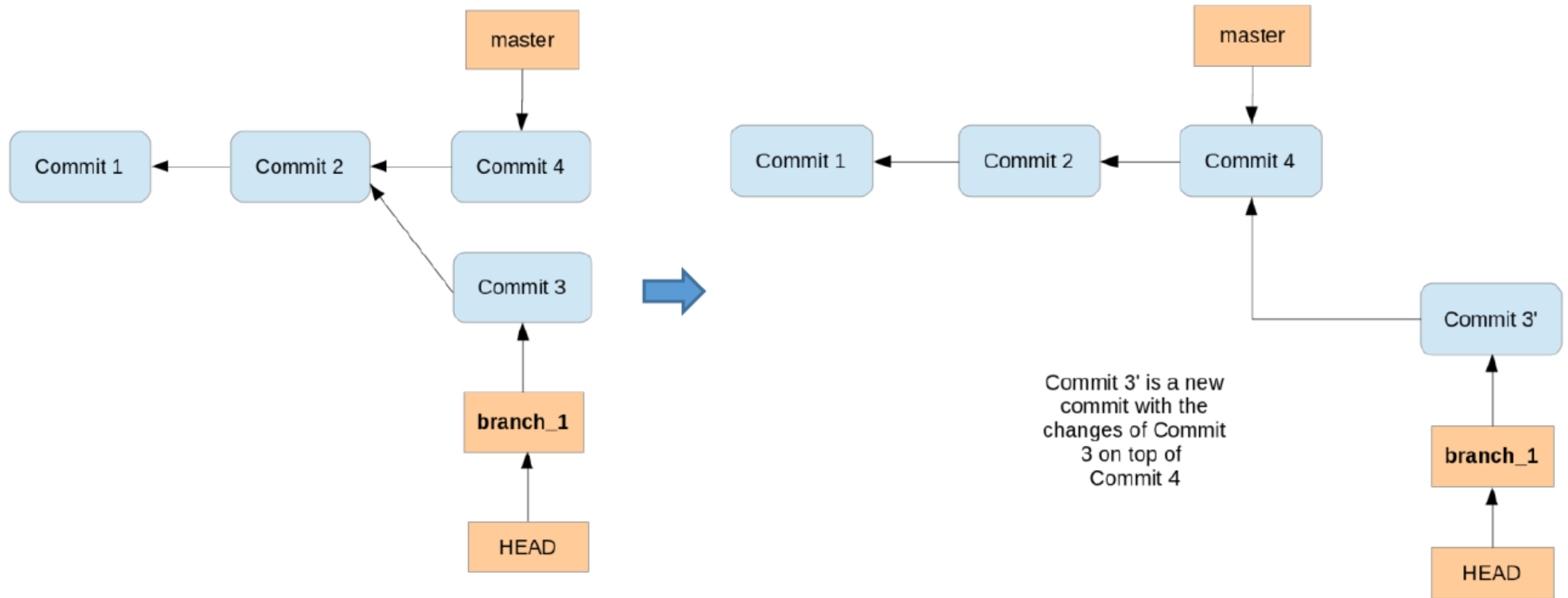
- **Merge commit:** Si se fusionan commits que no son sucesores directos de la rama actual, Git realiza una fusión de tres vías entre los últimos commits de las dos ramas, en función del predecesor común más reciente de ambas. Como resultado, se crea un merge commit en la rama actual, que combina los cambios respectivos de las dos ramas que se fusionan; este commit apunta a sus dos predecesores. Un merge commit es útil, por ejemplo, si interesa que quede reflejado en la historia que una funcionalidad se desarrolló en paralelo.



Fusiones de variantes

- **Rebase:** Esta estrategia ayuda a mantener una historia que resulte fácil de leer. Por ejemplo, al traer los cambios desde un repositorio remoto, puede ser preferible hacer un rebase a un merge commit. Cuando se hace un rebase de una rama A sobre otra rama B, el comando git toma los cambios introducidos por los commits de la rama A y los aplica en función de la HEAD de la rama B. Después de esta operación, los cambios en la rama B también están disponibles en la rama A. La ejecución del rebase crea un nuevo commit con los cambios de la rama A situados sobre la HEAD de la rama B. Realizar un rebase no crea un merge commit. El resultado final para el código fuente es el mismo que con un merge pero el historial de commits es más limpio; la historia parece ser lineal. Rebase se puede utilizar para ir actualizando con frecuencia una rama de funcionalidad F del repositorio local con los cambios que se van introduciendo en la rama maestra. Esto garantiza que esta rama F, cuyo desarrollo se puede prolongar en el tiempo, esté cerca de la HEAD de la rama master, hasta que finalmente se fusione con ella cuando acabe el desarrollo de la funcionalidad.

Fusiones de variantes



- **Cherry-pick:** Cherry-pick permite seleccionar uno o varios cambios de una rama y transferirlos a otra rama. Los cambios son introducidos en la segunda rama en un nuevo commit. El nuevo commit no apunta a su commit original, así que no han de realizarse cherry-pick a la ligera, ya que podría terminarse con varias copias del mismo cambio. Es común para copiar arreglos de bugs hechos sobre la rama master a otras ramas que se crearon a partir de esta.

Las etiquetas (tags) son muy útiles para hacer el seguimiento de los eventos importantes en el ciclo de vida de un proyecto. A medida que un proyecto avanza, va superando hitos; si se sigue una metodología ágil, se estarán generando versiones cada pocas iteraciones, y si se trabaja con una metodología tradicional se lanzarán versiones cada varios meses.

En cualquier caso, se necesita llevar el seguimiento del estado del repositorio en cada uno de esos hitos. Para ello se utilizan las etiquetas. Sirven para marcar un determinado momento en la historia del repositorio, así que nos podamos referir a él más adelante.

Una etiqueta es simplemente un nombre que señala un punto específico en la historia del repositorio, que puede ser el lanzamiento de una versión del proyecto o, simplemente, el momento en que determinado bug (error) fue corregido.

En Git, técnicamente una etiqueta es un puntero a un commit, y por tanto identifica de forma exclusiva una versión del repositorio. Tanto las ramas como las etiquetas son punteros, la diferencia entre ellas es que una rama se mueve cuando se crea un nuevo commit, mientras que una etiqueta siempre apunta al mismo commit. Además, las etiquetas pueden tener una marca de tiempo y un mensaje asociado con ellas.

El control de la configuración

El control de la configuración

Cuando se habla de software de gestión de configuración (SCM) a primera vista parece que se está hablando de control de versiones. Y esto es bastante acertado pues la gestión de la configuración se basa en gran medida en la existencia de un buen control de versiones, si bien el control de versiones es sólo una de las herramientas utilizadas.

La gestión de la configuración es un conjunto de prácticas de gestión de proyectos que permiten producir software de manera controlada y reproducible. Hace uso del control de versiones para lograr sus objetivos técnicos, pero también utiliza una gran cantidad de controles humanos y de comprobaciones para asegurar que el proyecto progresa adecuadamente de acuerdo con su ciclo de vida.

La ingeniería de software recomienda realizar el desarrollo de manera disciplinada. Las herramientas de control de versiones no garantizan un desarrollo razonable si cualquier miembro del equipo puede realizar los cambios que quiera e integrarlos en el repositorio sin ningún tipo de control; es necesario aplicar controles al desarrollo e integración de cambios.

Uso de GIT

GIT

Git es una de las herramientas de control de versiones más utilizadas. Entre sus características se encuentran:

- a) Es un SCV distribuido, gratuito y de código abierto.
- b) Puede utilizarse para proyectos pequeños y grandes.
- c) Es fácil de aprender y tiene un buen rendimiento.



Para ver cómo trabajar con GIT, haremos uso de GitHub que es un repositorio público y gratuito donde almacenar tu código.



Git conceptos básicos

Antes de empezar con un caso práctico, repasemos algunos conceptos básicos de GIT:

- **Commit:** identifica los cambios hechos.
- **Push:** sube los cambios hechos a una rama de trabajo tuya y/o de tu equipo remota.
- **Pull request:** solicitud que se hace al propietario del proyecto para realizar un cambio.
- **Pull:** actualizar su rama HEAD actual con los últimos cambios desde el servidor remoto.
- **Merge:** fusionar la rama en la que se está trabajando con la original.
- **Fetch:** descarga datos nuevos de un repositorio remoto, pero no integra ninguno de estos nuevos datos en tus archivos de trabajo.

Git conceptos básicos

Para saber cuál es el workflow para trabajar con Git hay que seguir ciertos pasos:

1. Crear una rama a partir de la rama principal (master).

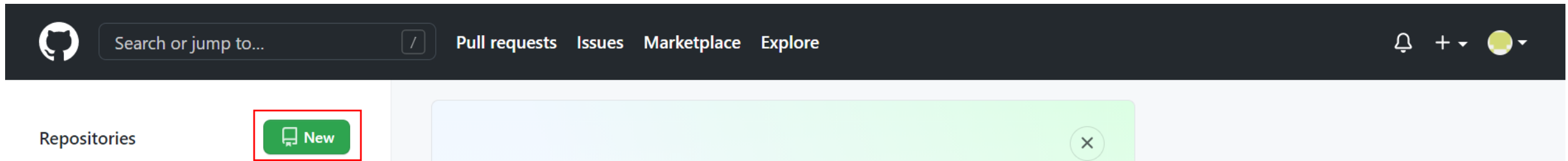
La regla número uno para trabajar con GIT es que **nunca, jamás, bajo ningún concepto**, hacer un commit y/o push sobre la rama principal, normalmente llamada master.

2. Una vez creada la rama se hace las modificaciones de código necesaria y se hace commit and push para subir tu rama al repositorio.
3. Cuando el código está subido a tu rama en el repositorio, se crea una pull request, añadiendo como revisores a las personas que se dedican a repasar las pull request en tu proyecto.

GIT se puede descargar gratuitamente de <https://git-scm.com/> pero la mayoría de los IDE, ya incluyen un cliente de GIT para poder conectar y trabajar con un repositorio como GitHub. Veremos como trabajar con un IDE, pero también indicaremos las directivas principales para trabajar directamente desde la consola.

Antes de trabajar con GitHub habrá que registrarse en la plataforma, una vez ahí crearemos un proyecto pulsando en new.

Recuerda al registrarte en Git Hub no usar tus datos personales.



Una vez hecho esto la creación de un nuevo repositorio es bastante intuitivo, y basta con seguir los pasos.

Habrà que elegir el nombre del repositorio y si el repositorio será público o privado

A screenshot of the 'Create a new repository' form on GitHub. The form has a title 'Create a new repository' and a subtitle 'A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.' Below this, there are two main sections: 'Owner' and 'Repository name'. The 'Owner' dropdown is set to 'mherrera-fp' and the 'Repository name' dropdown is set to 'test' with a green checkmark. Below these, there is a note: 'Great repository names are short and memorable. Need inspiration? How about upgraded-guide?'. Then, there is a 'Description (optional)' text input field. At the bottom, there are two radio button options: 'Public' (with a computer icon) and 'Private' (with a lock icon). The 'Private' option is selected with a blue dot. The 'Public' option description is 'Anyone on the internet can see this repository. You choose who can commit.' and the 'Private' option description is 'You choose who can see and commit to this repository.'

Además, desde el 13 de Agosto de 2021, GitHub incrementó su seguridad, imposibilitando conectarte a los repositorios usando la misma clave que da acceso a GitHub, por lo que tendremos que logarnos de otra forma. La más fácil es crear un token de acceso (**Personal access tokens**). Para ello haremos

1. Ir a GitHub Settings > Developer settings > [Personal access tokens website](#)
2. Haremos click en Generate new token (Arriba a la derecha)
3. Añadimos un nombre y un tiempo de expiración y generamos el token

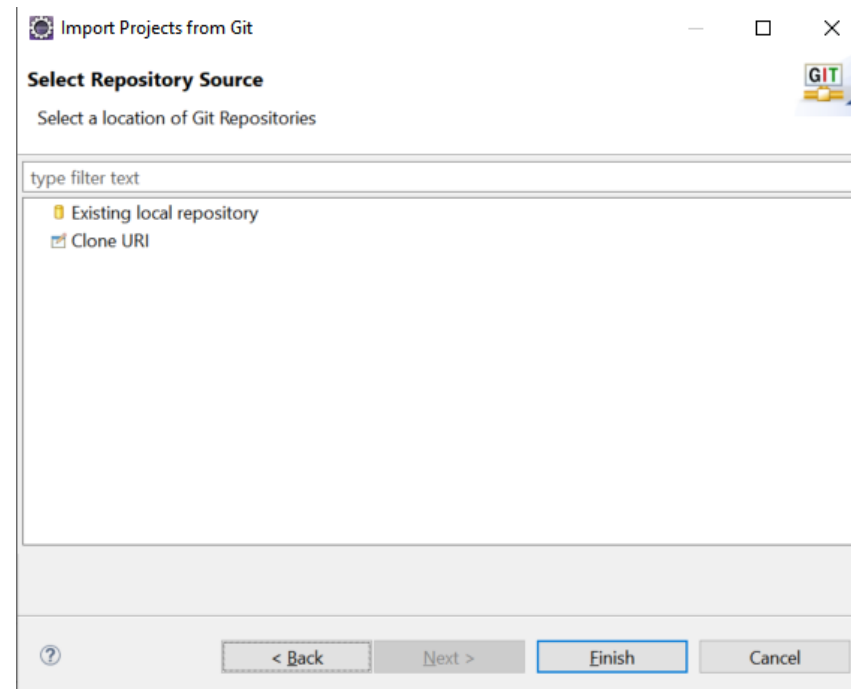
Actividad 2

Crea un repositorio en GitHub añadiendo un README en el proceso de creación

Eclipse GIT

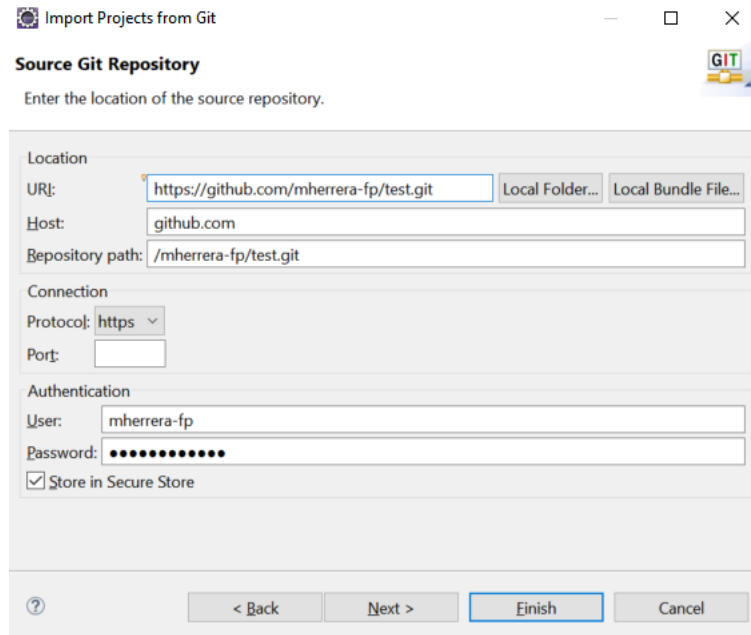
Una vez creado el repositorio de Git, podremos conectar GitHub haciendo uso de algún tipo de herramienta de sincronización de código, incluso podemos utilizar nuestro IDE para trabajar con GIT, con eclipse podemos usar Eclipse Git.

Vamos a ver dos formas de integrarnos, la primera para un proyecto ya creado en el repositorio, por lo que deberemos descargarlo del repositorio, para ello pulsaremos en File > Import > Projects from Git > Clone URI

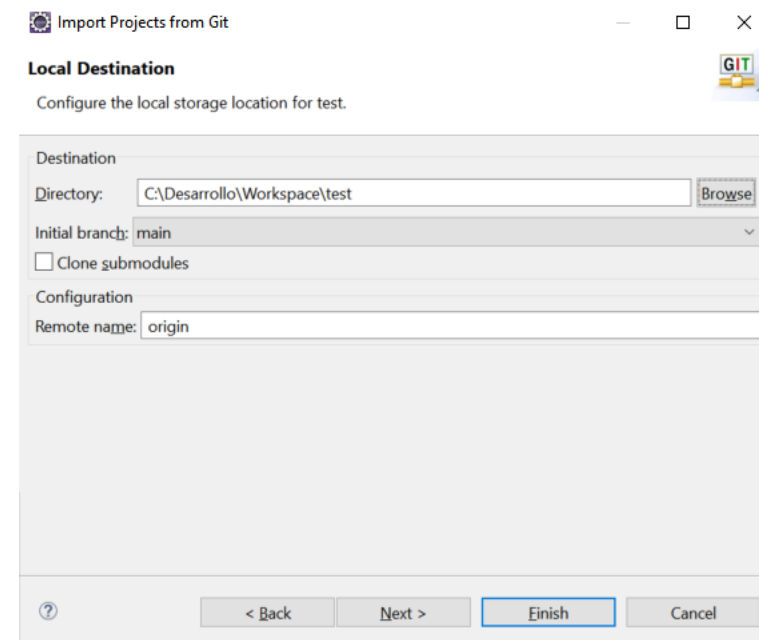


Eclipse GIT

Cuando hemos pulsado en Clone URI se nos mostrará la pantalla para añadir los datos de GIT, URI, username y password



The screenshot shows the 'Import Projects from Git' dialog box with the 'Source Git Repository' tab selected. The dialog has a title bar with a question mark icon and standard window controls. The main content area is divided into sections: 'Location' with fields for 'URI' (containing 'https://github.com/mherrera-fp/test.git'), 'Host' (containing 'github.com'), and 'Repository path' (containing '/mherrera-fp/test.git'); 'Connection' with a 'Protocol' dropdown set to 'https' and an empty 'Port' field; and 'Authentication' with 'User' (containing 'mherrera-fp') and 'Password' (masked with dots) fields, and a checked 'Store in Secure Store' checkbox. At the bottom are buttons for '< Back', 'Next >', 'Finish' (highlighted with a blue border), and 'Cancel'.



The screenshot shows the 'Import Projects from Git' dialog box with the 'Local Destination' tab selected. The dialog has a title bar with a question mark icon and standard window controls. The main content area is divided into sections: 'Destination' with a 'Directory' field (containing 'C:\Desarrollo\Workspace\test') and a 'Browse' button; 'Initial branch' (a dropdown menu set to 'main'); a checked 'Clone submodules' checkbox; and 'Configuration' with a 'Remote name' field (containing 'origin'). At the bottom are buttons for '?', '< Back', 'Next >', 'Finish' (highlighted with a blue border), and 'Cancel'.

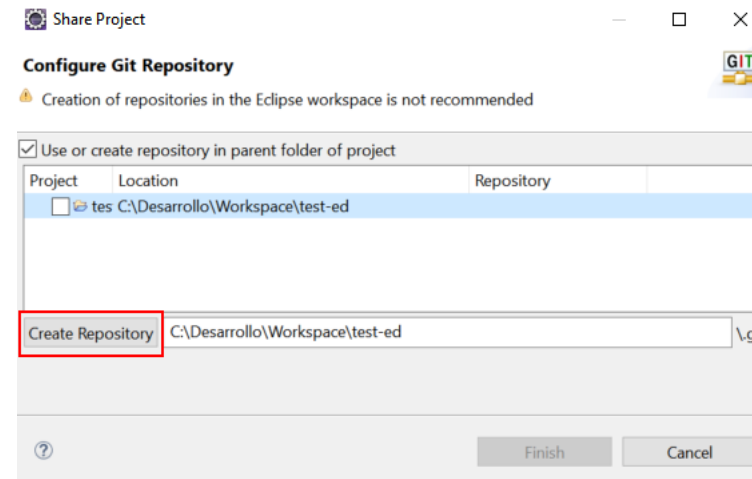
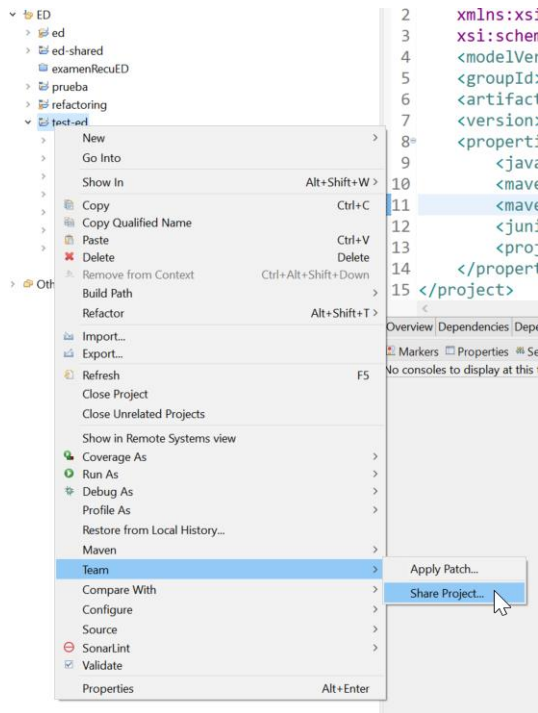
Seleccionamos la rama a descargar y el directorio dónde queremos que se descargue el repositorio. Cuando hayamos seleccionado la carpeta dónde se descargará nuestro repositorio, tendremos que elegir como importarlo, la mejor opción, suele ser, usar el wizard del proyecto para seleccionar el tipo de nuestro proyecto.

Actividad 3

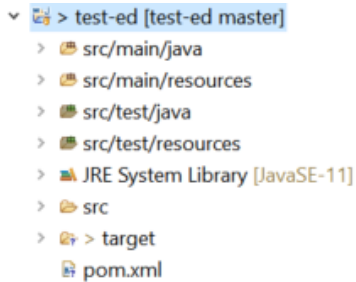
Descarga el código creado en el repositorio de GIT.

Otra opción es tener ya un código creado y querer repositarlo, para esta opción sería mejor añadir el código al repositorio que hemos creado. El primer paso que vamos a realizar es pulsar botón derecho sobre el proyecto e ir a la opción de Team (Equipo) y pulsar sobre share (compartir).

Realizada esta operación nos aparece una nueva ventana donde elegimos que carpeta será nuestro repositorio.



Pulsamos finalizar en las ventanas que nos aparecen y los iconos del proyecto de eclipse cambiarán.

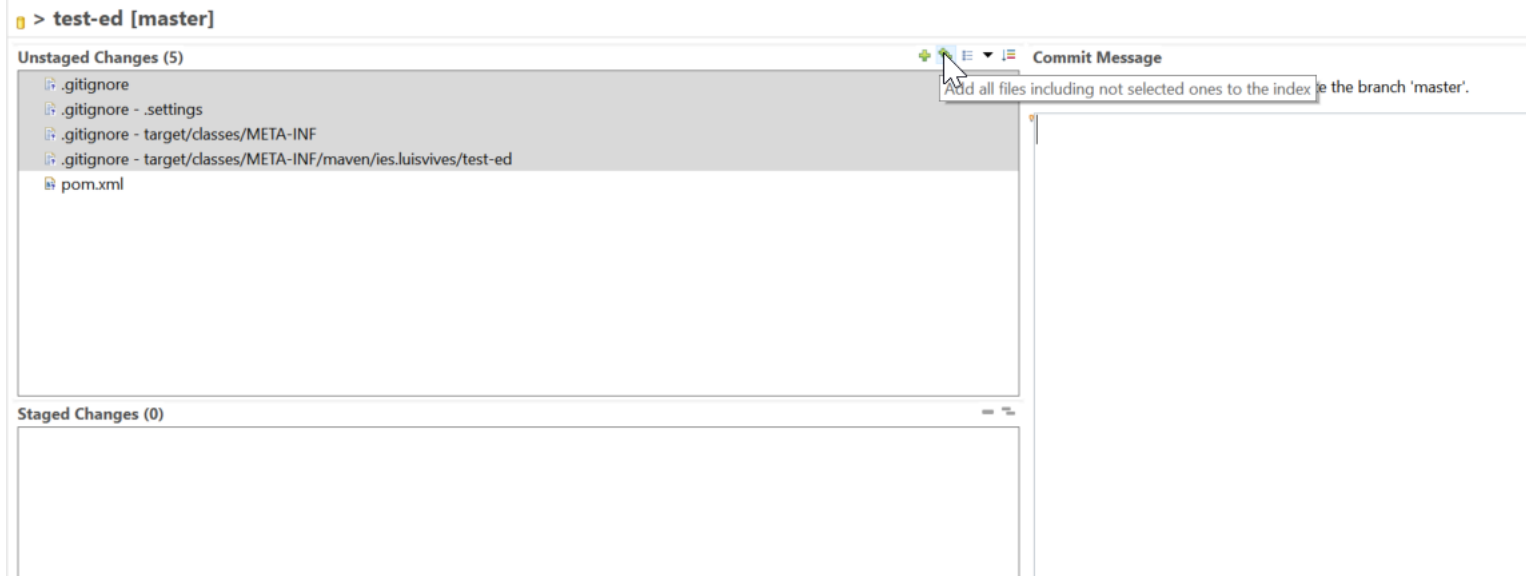


Ya tenemos a nuestra disposición todos los ficheros para añadirlos al repositorio. Para empezar a enviar el contenido es necesario hacer un commit. Team > Commit

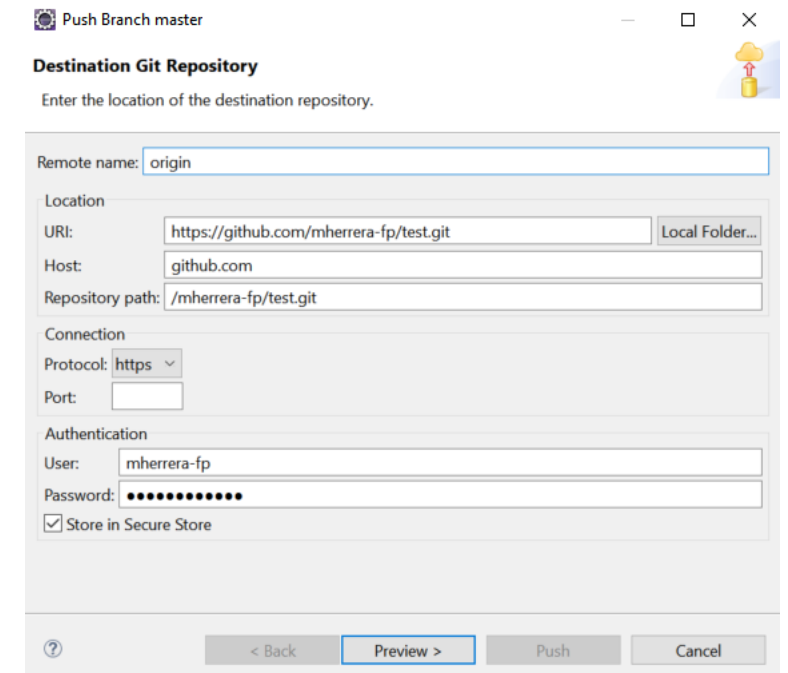
Previamente podemos ignorar todos los ficheros que no queremos subir, para hacer esto seleccionamos los ficheros que no queremos subir y con botón derecho elegimos ignore

Eclipse Git

Cuando ya tenemos hecho esto solo tenemos que añadir los ficheros a comitar. Y añadir un mensaje al commit.



Luego deberemos hacer commit



Eclipse Git

Una vez hecho commit, podremos hacer nuestro primer push, dónde seleccionaremos el repositorio de GitHub

 Push Branch master

Destination Git Repository

Enter the location of the destination repository.

Remote name:

Location

URI: [Local Folder...](#)

Host:

Repository path:

Connection

Protocol:

Port:

Authentication

User:

Password:

☒ Store in Secure Store

[?](#) [< Back](#) [Preview >](#) [Push](#) [Cancel](#)

 Push Branch master

Push to branch in remote

Select a remote and the name the branch should have in the remote.

Source:

 master  7cebe34 Test

Destination:

Remote:

Branch:

☒ Configure upstream for push and pull

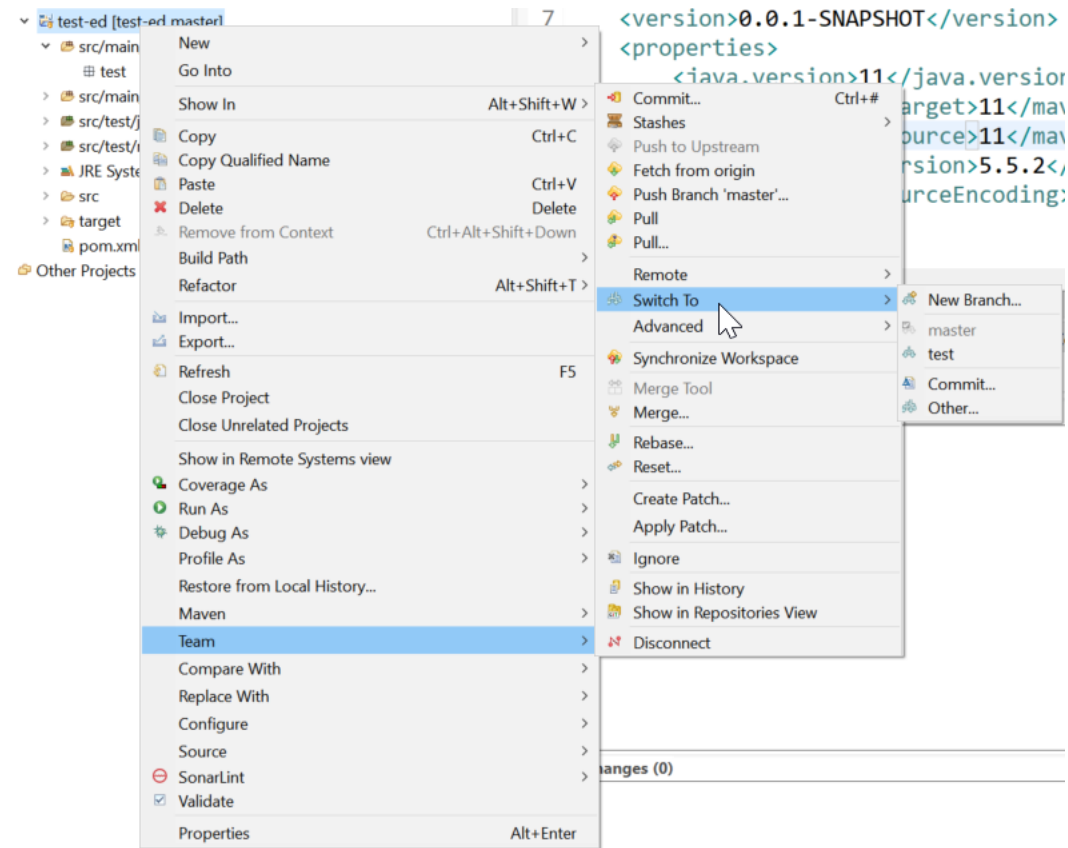
When pulling:

☐ Force overwrite branch in remote if it exists and has diverged

[Show advanced push dialog](#)

[?](#) [< Back](#) [Preview >](#) [Push](#) [Cancel](#)

Para cambiar de rama, lo haremos con la opción de Switch to



Actividad 4

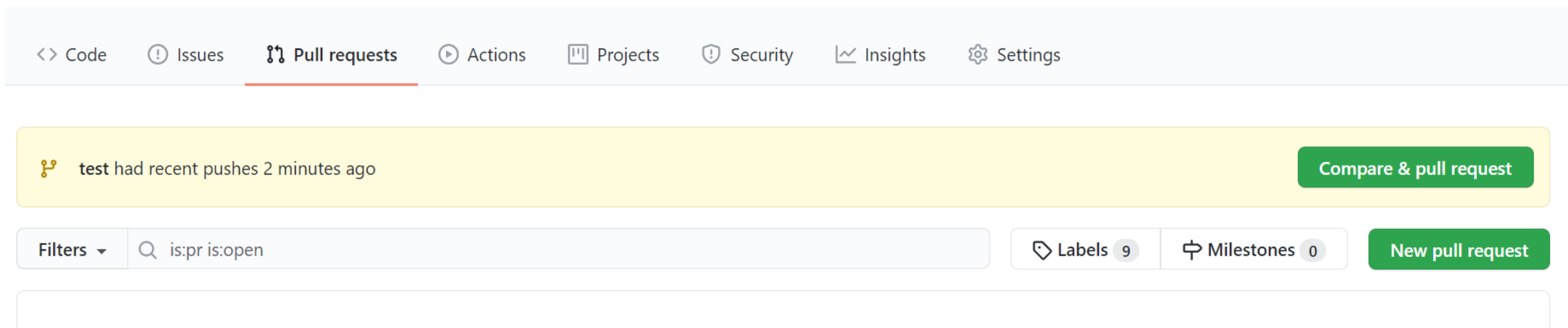
Crea un proyecto Maven y súbelo a un nuevo repositorio de GitHub.

Actividad 5

Crea una rama haz modificaciones y sube la rama al GitHub.

Pull Request

Para hacer un cambio en master hay que pedir una pull request. En GitHub hay que acceder a la pestaña de pull request y solicitar una nueva pull request



Actividad 6

Modifica un proyecto de git y crea una pull request.

Principales comandos de GIT

GIT

Hoy en día, todo lo que se puede hacer con comandos por la consola de GIT, está implementado en los IDE, pero no viene mal conocer alguno de estos comandos.

Tenéis una hoja con los principales comandos [github-git-cheat-sheet.pdf](#) en el aula virtual y una hoja de ayuda [GitKraken-Git-Basics-Cheat-Sheet.pdf](#)

Refactorización

Refactorización

La refactorización es una técnica para la reestructuración de un bloque existente de código fuente, alterando su estructura interna sin cambiar su comportamiento externo. Consiste en realizar una serie de pequeñas transformaciones que preserven el comportamiento.

Cada transformación (llamada “refactorización”), supone poco, pero una secuencia de transformaciones puede producir una importante reestructuración. Dado que cada refactorización es pequeña, es poco probable que introduzca errores. El sistema se mantiene en pleno funcionamiento después de cada pequeña refactorización, reduciéndose las posibilidades de que un sistema puede quedar seriamente perjudicado durante la reestructuración.

Existe una estrecha relación entre la refactorización y la utilización de patrones. A menudo, la mejor manera de utilizar los patrones es refactorizar gradualmente el código para introducir el patrón una vez que se observa que es aplicable.

Refactorización

Muchos proyectos de desarrollo de software incluyen bases de datos relacionales y necesitan de la refactorización tanto como el código fuente. Refactorizar bases de datos es algo más complicado, ya que no sólo hay que cambiar el código y el esquema, también hay que migrar los datos; sin embargo, utilizar la técnica bien es esencial para aplicar enfoques de diseño evolutivos en los entornos de base de datos. Aunque es excesivo referirse a HTML como lenguaje de programación, el concepto de refactorización se aplica muy bien a HTML. Un HTML mal estructurado puede ser una barrera significativa para el mantenimiento de un sitio web con actualizaciones periódicas. Refactorizar HTML permite mantener sitios ágiles, que incorporen fácilmente los cambios emergentes en las tecnologías web.

La refactorización mejora el diseño de software: sin refactorización, el diseño del programa decaerá. Cuando la gente cambia el código —cambios para alcanzar los objetivos a corto plazo o cambios realizados sin una comprensión completa del diseño del código— el código pierde su estructura. Se hace más difícil ver el diseño mediante la lectura del código.

Refactorización

La refactorización es algo así como poner en orden el código; se realiza trabajo para eliminar los fragmentos que no están realmente en el lugar correcto. La pérdida de la estructura de código tiene un efecto acumulativo; cuanto más difícil es ver el diseño en el código, más difícil es preservarlo, y más rápidamente se desintegra. La refactorización regular del código ayuda a mantener su forma.

Cada vez que se va a hacer refactorización, el primer paso ha de ser siempre el mismo: construir un sólido conjunto de pruebas para esa sección de código. Las pruebas son esenciales porque, aunque se realicen refactorizaciones estructuradas para evitar en la medida de lo posible la introducción de errores, es propio del ser humano cometer errores.

Refactorización

La refactorización nunca modificará el comportamiento de un método, una clase o más genérico del software. Para refactorizar un programa, habrá que realizar una serie de cambios internos reestructurando componentes, pero siempre teniendo en cuenta que su comportamiento debe permanecer inalterado. Ventajas de refactorizar:

- a) Ayudará al programador a encontrar errores de una forma más rápida y sencilla. Muchas veces, una estrategia equivocada al programar implica errores y fallos internos a la larga.
- b) Ayudará a programar más rápido. Seguramente, al encontrar menos errores, se perderá menos tiempo en corregir y arreglar código.
- c) Los programas se leerán e interpretarán de una manera más fácil.
- d) Los diseños serán más robustos.
- e) Los programas tendrán más calidad.
- f) Se evitará duplicar la lógica de los programas.

Patrones de refactorización

Patrones de refactorización

Extract method o reducción lógica. Este tipo de refactorización es una de las más sencillas y típicas. Cuando se escriba código, lo que hay que crear son métodos que hagan lo que dicen. Muchas veces, los métodos cortos tienen muchas ventajas:

- Realizan tareas específicas concretas, con lo cual pueden ser invocados por otros métodos de forma más sencilla (reutilización).
- Permiten ir creando otros métodos con un nivel de complejidad mayor.

```
void printCuenta(String nombre, double cantidad){  
    printLogo();  
    System.out.println("nombre:"+nombre);  
    System.out.println("cantidad:"+cantidad);  
}
```



```
void printCuenta2(String nombre, double cantidad){  
    printLogo();  
    printDetalles(nombre, cantidad);  
}  
void printDetalles(String nombre, double cantidad){  
    System.out.println("nombre:"+nombre);  
    System.out.println("cantidad:"+cantidad);  
}
```

Patrones de refactorización

Métodos inline o código embebido. Si lo que se pretende es escribir métodos y código que muestren la intención del código, muchas veces, ocultar la lógica del programa factorizando el código puede ser mala idea. En el ejemplo anterior, se observa que el código queda más simple y fácil de entender eliminando la función, puesto que se reduce su complejidad y es más legible. De hecho, no debería utilizarse este tipo de formas de programar en Java cuando hay métodos que sobrecargan métodos de clases base.

```
int rondaGratis(){  
    return (masde5cañas())?2:1;  
}  
boolean masde5cañas(){  
    return numerodecañas > 5;  
}
```



```
int rondaGratis(){  
    return (numerodecañas > 5)?2:1;  
}
```

Otro ejemplo sería utilizar una variable temporal solamente para realizar una acción y luego devolver esa variable temporal.

```
double preciobase = pedido.preciobase();  
return (preciobase > 100);
```



```
return (pedido.preciobase()>100);
```

Patrones de refactorización

En el siguiente ejemplo, desea realizarse un descuento al cliente. Si la cuenta supera una determinada cantidad, pagará un 90% (descuento de un 10%), en caso contrario, el descuento solamente será de un 5%. Podemos observar que se utiliza una variable temporal para calcular el precio base de una cuenta. Muchas veces, en vez de utilizar una variable temporal, es mejor utilizar un nuevo método con un nombre suficientemente explicativo para determinar el precio base. Este método podrá ser empleado por otros métodos de la clase, con lo cual su utilidad se incrementaría.

```
double preciobase = cantidad*preciounitario;  
if (preciobase > 1000)  
    return preciobase*0.90;  
else  
    return preciobase*0.95;
```



```
if (precioBase() > 1000)  
    return preciobase*0.90;  
else  
    return preciobase*0.95;  
...  
double precioBase(){  
    return cantidad*preciounitario;  
}
```

Patrones de refactorización

Otra opción sería querer utilizar una variable sin tener que crear un método para factorizarla. En este caso, se recomienda utilizar variables final. Utilizar una variable de tipo *final* permite asignarle un valor y previene de una doble instanciación porque el compilador va a mostrar el error. Por ejemplo, el siguiente código mostrará un error:

```
final double preciobase;  
preciobase = cantidad*precio;  
preciobase = cantidad*precio 0.9;
```

Con lo cual, el método podría ser redefinido de la siguiente forma:

```
final int preciobase = cantidad*precio;  
final double descuento;  
if (preciobase > 1000) descuento = 0.9;  
else descuento = 0.95;  
return preciobase*descuento;
```


Variables autoexplicativas. Muchas veces, los programadores se obcecán en escribir complejas líneas de código cuando se tienen recursos para hacer el código más legible. Por ejemplo, es fácil encontrar sentencias de código como la siguiente:

```
if ((idioma.toUpperCase().indexOf("RUS") > -1) &&  
    idioma.toUpperCase().indexOf("ALE") > -1) &&  
    (nivelingles > 0)) {  
    // MENSAJES EN INGLES  
}
```

Utilizando variables autoexplicativas correctamente definidas (obsérvese que se han definido como tipo final), el código resultaría mucho más legible y sencillo. Véase el código anterior corregido:

```
final boolean esRuso= idioma.toUpperCase().indexOf( "RUS") > -1;  
final boolean esAleman = idioma.toUpperCase().indexOf("ALE") > -1;  
final boolean ingles = nivelingles > 0;  
if (esRuso && esAleman && ingles) {  
    // MENSAJES EN INGLES  
}
```

Mal uso de variables temporales. Imagínese el siguiente código:

```
double tmp = pi radio radio;  
System.out.println (tmp);  
tmp = 2 pi radio;  
System.out.println (tmp);
```

En realidad, nuestro código debería haber tenido este aspecto:

```
final double area = pi radio radio;  
System.out.println (area);  
final double perimetro = 2 pi radio;  
System.out.println (perimetro);
```

¿Por qué se hace esto? Porque una variable temporal, generalmente, se utiliza para ser instanciada o establecida solamente una vez por lo que, en caso de que tenga que cambiarse su valor varias veces durante la ejecución del método, hay que plantearse crear una segunda variable temporal. En el caso anterior, se ve claramente que habría que utilizar dos variables temporales en vez de una. Además, para evitar la doble instanciación, se declaran las variables como final.

Patrones de refactorización

Es preciso tener cuidado con los parámetros. Imagínese que tiene el siguiente código en su programa (las horas extras computan 1,5 la hora trabajada):

```
int salario(int horas, int horasextra, int salariobase){  
    horas = horas + horasextra*1.5;  
    ...  
}
```

En realidad, el código debería haber tenido este aspecto:

```
int salario(int horas, int horasextra, int salariobase){  
    final int horastrabajadas = horas + horasextra * 1.5;  
    ...  
}
```

¿Por qué se hace esto? Porque un parámetro no debería ser modificado, salvo que haya una razón para ello. Generalmente, de los parámetros, suele interesar su valor y no se desea que este cambie una vez que se haya invocado el método. En el caso anterior, el valor del parámetro horas no tiene sentido que sea modificado. Salvo razón obvia, es mejor no cambiar el valor de los parámetros durante la ejecución de un método.

Cambiar algoritmos. Siempre hay una manera más fácil de hacer una cosa. Cuando creas que hay una mejor manera de realizar algo, sustituye. La factorización trata de simplificar cosas más complejas en otras más sencillas y fáciles de entender.

En el caso de que tenga que sustituirse un algoritmo, trata de hacerlo de tal manera que funcione igual o mejor que antes. Las pruebas de regresión pueden ir bien para comparar resultados de algoritmo antiguo y nuevo.

Siempre que puedas utilizar o crear una librería, hazlo

Patrones de refactorización

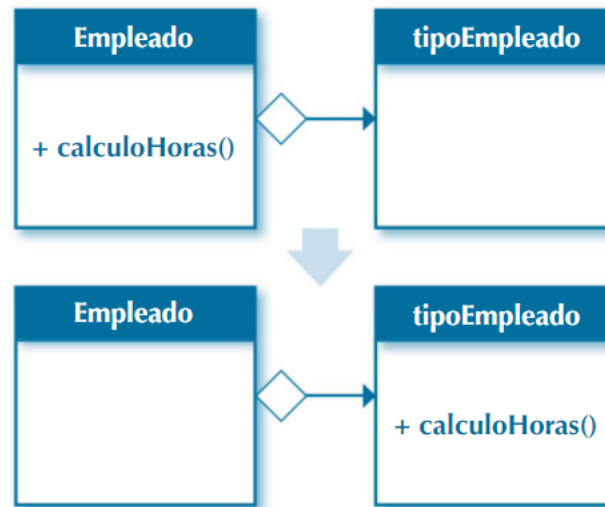
Imagínese que se tiene el siguiente código:

```
String BuscaAnimal(String[] animales){
    for (int i = 0; i < animales.length; i++) {
        if (animales[i].equals ("Perro")){
            return "Perro";
        }
        if (animales[i].equals ("Tortuga")){
            return "Tortuga";
        }
        if (animales[i].equals ("Loro")){
            return "Loro";
        }
    }
    return "No encontrado";
}
```

El resultado de una refactorización que mejore la legibilidad y eficiencia del código anterior sería la siguiente:

```
String BuscaAnimal2(String[] animales){
    for (String animale: animales) {
        if (animale.equals("Perro")) {
            return "Perro";
        }
        if (animale.equals("Tortuga")) {
            return "Tortuga";
        }
        if (animale.equals("Loro")) {
            return "Loro";
        }
    }
    return "No encontrado";
}
```

Mover métodos entre clases. Muchas veces, se tienen clases complejas y otras que realizan pocas tareas. Cuando se tengan en un esquema de diseño unas clases sobreexplotadas con una gran cantidad de métodos y estados internos y otras más ligeras, hay que replantearse repartir la carga de trabajo. Si hay algo que no funciona, repartir funciones es una estrategia para acotar el problema. Imagínese que tiene un esquema de clases como el de la siguiente imagen (ojo, el ejemplo se ha simplificado al máximo).



Patrones de refactorización

En ocasiones, es necesario repartir carga de trabajo y mudar o trasladar métodos entre clases para que la carga sea más homogénea en todo el esquema. Véase un ejemplo de cómo mudar o trasladar un método de una clase a otra.

```
public class tipoEmpleado {  
    private String tipo;  
    private double horabase;  
    tipoEmpleado(String t,double h){  
        this.tipo=t; this.horabase=h;  
    }  
    public String getTipo(){return tipo;  
    }  
    public double getHorabase(){return horabase;  
    }  
}
```

```
public class Empleado {  
    private int horas;  
    private int horasextra;  
    private tipoEmpleado tipo;  
    public double calculoHoras(){  
        if (tipo.getTipo().equals("Supervisor")) {  
            return horas + horasextra * 1.40;  
        }  
        if (tipo.getTipo().equals("Dependiente")) {  
            return horas + horasextra * 1.75;  
        }  
        return horas + horasextra * 1.5;  
    }  
    public double getsueldo(){  
        return tipo.getHorabase()*calculoHoras();  
    };  
}
```

Patrones de refactorización

De las clases anteriores, puede deducirse que la clase Empleado crecerá hasta hacerse muy voluminosa, mientras que la clase tipoEmpleado seguirá siendo muy simple. La mejor forma de hacer los programas más legibles y fáciles de mantener es factorizar y racionalizar funciones.

El resultado de esta racionalización sería el siguiente:

```
public class tipoEmpleado {
    private String tipo;
    private double horabase;
    tipoEmpleado(String t,double h){
        this.tipo=t; this.horabase=h;
    }
    public String getTipo(){
        return tipo;
    }
    public double getHorabase(){
        return horabase;
    }
    public double calculoHoras(int horas,int horasextra){
        if (tipo.equals("Supervisor")) {
            return horas + horasextra * 1.40;
        }
        if (tipo.equals("Dependiente")) {
            return horas + horasextra * 1.75;
        }
        return horas + horasextra * 1.5;
    }
}
```

```
public class Empleado {
    private int horas;
    private int horasextra;
    private tipoEmpleado tipo;
    public double getsueldo(){
        return tipo.getHorabase()*tipo.calculoHoras(horas,horasextra);
    };
}
```

Hay que tener cuidado con este tipo de movimientos porque no siempre es fácil realizar cambios de racionalización siguiendo una arquitectura dada. Por lo que este tipo de cambios deben ser pensados y analizados en el conjunto. Por ejemplo, la mejor opción en el ejemplo sería no tener lógica de negocio en algo que parece una Entidad y llevar esa lógica a un servicio, por ejemplo, EmpleadoService.

Mover variables miembro entre clases. Mover métodos y campos es la base de la refactorización. Imagínese que, en el ejemplo anterior, el precio de la hora base no depende del tipo de empleado y va a ser utilizado por muchos métodos de la clase empleado. En vez de tenerlo en la clase tipoEmpleado, seguramente, sea mejor embeberla en la clase empleado. El resultado sería tener en la clase Empleado otra variable más y eliminar dicha variable de la clase tipoEmpleado:

```
public class Empleado {  
    private int horas;  
    private int horasextra;  
    private double horabase;  
    private tipoEmpleado tipo;  
    ...  
}
```

Autoencapsular campos (getters y setters). Una buena práctica cuando una variable va a ser utilizada de forma asidua es autoencapsularla en la clase creando sus getters y setters. En la clase tipoEmpleado, el resultado de tener setters y getters para la variable tipo sería el Siguiente:

```
private String tipo;  
public String getTipo(){ return this.tipo;}  
public void setTipo(String t){this.tipo = t;}
```

Extraer clases. ¿Tiene en una clase código que hace más de lo que debería? Si tiene en una clase mucho código, es que no se ha diseñado correctamente el esquema de clases. Por ejemplo, véase la siguiente clase:

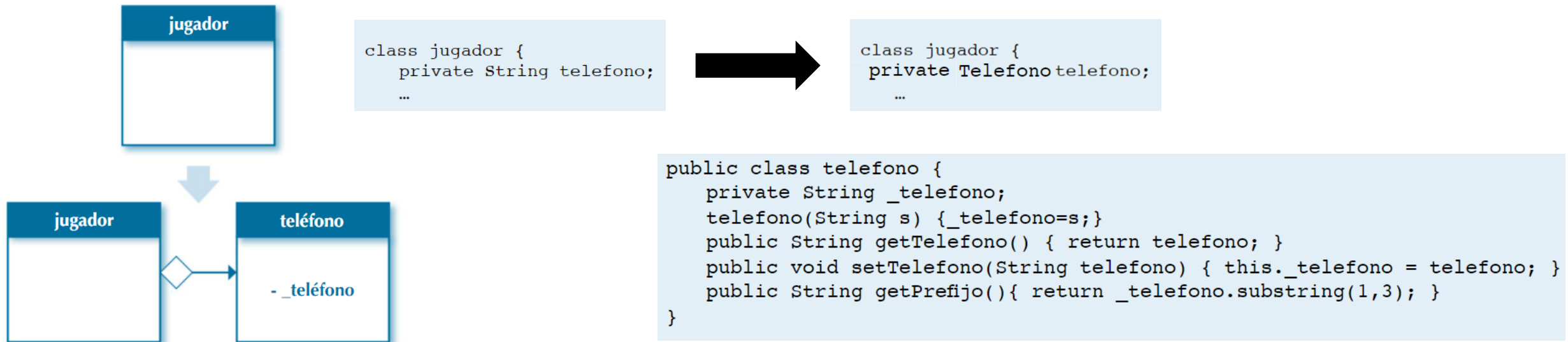
```
public class empleado2 {
    private int horas;
    private int horasextra;
    private final double horabase;
    private String tipo;
    public String getTipo(){return this.tipo;
    }
    public void setTipo(String t){this.tipo = t;
    }

    empleado2(String t,double h){
        this.tipo=t;
        this.horabase=h;
    }
    public double getHorabase(){return horabase;
    }
    public double calculoHoras(int horas,int horasextra){
        if (tipo.equals("Supervisor")) {
            return horas + horasextra * 1.40;
        }
        if (tipo.equals("Dependiente")) {
            return horas + horasextra * 1.75;
        }
        return horas + horasextra * 1.5;
    }
    public double getsueldo(){
        return getHorabase() calculoHoras(horas,horasextra);
    }
}
```

Puede adivinarse que todavía habría que añadir más funcionalidad y su volumen ya es grande. Si continúan añadiéndose métodos, getters, setters, etc., la clase crecerá hasta el infinito. La solución es extraer parte de la funcionalidad en otra clase. De esa forma, la clase será más fácil de mantener, ampliar y modular.

Patrones de refactorización

Reemplazar valores con objetos. Muchas veces, al comienzo del diseño, se toman decisiones que pueden evitar un crecimiento o desarrollo futuro de la aplicación. En ocasiones, se tienen datos que son tan simples que no tienen suficiente entidad para crear una clase. Pero, cuando se desarrolla más la aplicación, se observa que ese número de teléfono, por ejemplo, que se creía que era tan simple, necesitará formatearse de una manera determinada, extraer el prefijo, determinar si es móvil o fijo, etc. Cuando se vea que esto puede ocurrir, hay que modificar la clase y extraer el campo creando una nueva clase. Imagínese que tiene la clase Jugador, que es parecida a la siguiente:



Uso de constantes. Muchas veces, cuando el programador cree que solo va a utilizar un valor una vez, utiliza una variable y se olvida de declarar una constante. Para ser un programador eficiente, es mejor utilizar constantes siempre que se pueda. Sería preciso cambiar los métodos como este:

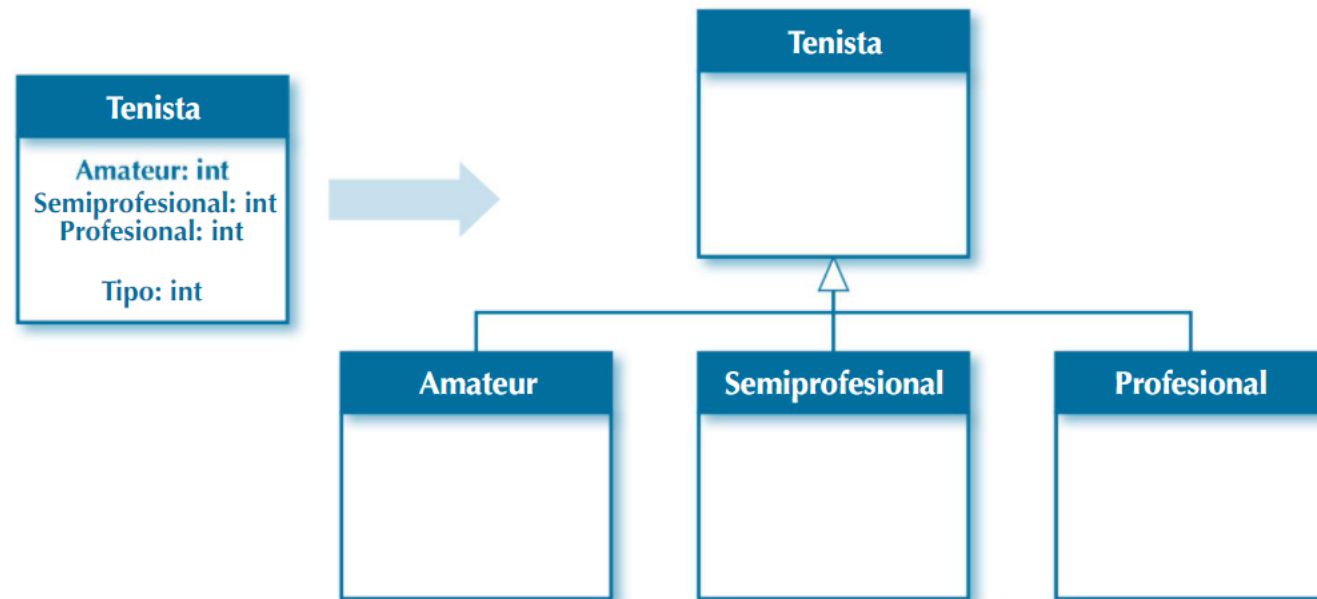
```
double descuento(double unidades, double preciounitario) {  
    return unidades*0.95*preciounitario;  
}
```



```
static final double DESCUENTO_DIRECTO = 0.95;  
double descuento(double unidades, double preciounitario) {  
    return unidades * DESCUENTO_DIRECTO * preciounitario;  
}
```

Patrones de refactorización

Reemplazar tipos de objeto con subclases. Muchas veces, se utilizan variables internas para clasificar objetos. Este tipo de programación implica que, dentro del código, existan muchas sentencias condicionales, ya sea de tipo if o de otro tipo. En este caso concreto, dependiendo del tipo de tenista, el código tendrá que ejecutar una cosa u otra. Este tipo de programación no tiene sentido en una POO, luego, para refactorizar esta situación, lo mejor es cambiar toda la estructura condicional con polimorfismo. Se crearán clases derivadas y, mediante la herencia y el polimorfismo, se generará un código más limpio y fácil de mantener.



Refactorización en Eclipse

HERRAMIENTAS

Refactorización en Eclipse

Eclipse tiene distintos métodos de refactorización. Dependiendo de sobre qué mostremos el menú de refactorización, nos ofrecerá unas u otras opciones. Para refactorizar pulsaremos click derecho sobre el nombre del elemento deseado, y desplegaremos la opción **Refactor** del menú contextual.

CLASE	
Rename...	Alt+Shift+R
Move...	Alt+Shift+V
Extract Interface...	
Extract Superclass...	
Use Supertype Where Possible...	
Pull Up...	
Push Down...	
Extract Class...	
Generalize Declared Type...	
Infer Generic Type Arguments...	

MÉTODO	
Rename...	Alt+Shift+R
Move...	Alt+Shift+V
Change Method Signature...	Alt+Shift+C
Inline...	Alt+Shift+I
Extract Interface...	
Extract Superclass...	
Use Supertype Where Possible...	
Pull Up...	
Push Down...	
Extract Class...	
Introduce Parameter Object...	
Introduce Indirection...	
Infer Generic Type Arguments...	

ATRIBUTO	
Rename...	Alt+Shift+R
Move...	Alt+Shift+V
Extract Interface...	
Extract Superclass...	
Use Supertype Where Possible...	
Pull Up...	
Push Down...	
Extract Class...	
Encapsulate Field...	
Generalize Declared Type...	
Infer Generic Type Arguments...	

Los métodos de refactorización nos permiten plantear casos y previsualizar las posibles soluciones que se nos ofrecen. Podemos seleccionar diferentes elementos para mostrar su menú de refactorización (una clase, una variable, método, bloque de instrucciones, expresión, etc). A continuación, se muestran algunos de los métodos más comunes.

Refactorización en Eclipse

- **Rename:** Es la opción empleada para cambiar el identificador a cualquier elemento (nombre de variable, clase, método, paquete, directorio, etc). Cuando lo aplicamos, se cambian todas las veces que aparece dicho identificador.
- **Move:** Mueve una clase (fichero.java) de un paquete a otro y se cambian todas las referencias. También se realiza la misma operación arrastrando la clase de un paquete a otro en el explorador de eclipse.
- **Extract Constant:** Convierte un número o cadena literal en una constante. Se puede ver donde se realizarán los cambios, y también el estado antes y después de refactorizar. Después, todas las apariciones de esa cadena se sustituyen por el nombre de la constante. Esto se utiliza modificar el valor en un solo lugar.
- **Extract Local Variable:** Convierte un número o cadena literal en una variable de ámbito local. Si esa misma cadena de texto existe fuera del bloque o del método, no se aplica el cambio. Parecido al patrón anterior, pero para aplicar dentro de método o bloques de código entre llaves { }.

Refactorización en Eclipse

- **Convert Local Variable to Field:** Convierte una variable local en un atributo privado de la clase. Después de aplicar el patrón de refactorización, todos los usos de la variable local se sustituyen por el atributo.
- **Extract Method:** Convierte un bloque de código en un método, a partir de un bloque cerrado por llaves { }. Eclipse ajusta los parámetros y el retorno del método. Es muy útil cuando detectamos bad smells en métodos muy largos, o en bloques de código que se repiten.
- **Change Method Signature:** Permite cambiar el nombre del método y los parámetros que recibe. Se actualizarán todas las dependencias y llamadas al método dentro del proyecto actual.

Refactorización en Eclipse

- **Inline:** Nos permite ajustar una referencia a una variable o método en una sola línea de código.
- **Extract Interface:** Este patrón de refactorización nos permite seleccionar los métodos de una clase para crear una Interface. Una Interface es una plantilla que define los métodos acerca de lo que puede hacer una clase. Define los métodos de una clase (Nombre, parámetros y tipo de retorno) pero no los desarrolla.
- **Extract Superclass:** Permite crear una superclase (clase padre) con los métodos y atributos que seleccionemos de una clase concreta. Lo usamos cuando la clase con la que trabajamos podría tener cosas en común con otras clases, las cuales serían también subclases de la superclase creada.

Actividad 7

Dado el siguiente el proyecto subido en el aula virtual descárgalo y refactoriza el método main.

El método main solo puede tener la creación del objeto DrinkMachine y la llamada al método start.

Actividad 8

Dado el siguiente el proyecto subido en el aula virtual descárgalo y refactoriza el método main.

El método main solo puede tener la llamada al método start de la clase Calificaciones.

Documentación

Durante el proceso de elaboración de software, se genera mucha documentación. Los muchos documentos de innumerables programadores, analistas, jefes de proyecto, etc., dejan claro que es mejor no generar ningún tipo de documentación que generar documentación inservible o poco útil.

Como se ha dicho anteriormente, durante el proyecto, se genera mucha documentación, pero también antes y después. La documentación de un programa o producto va dirigida a muchas personas como programadores, usuarios, personas que van a testear la aplicación, etc. Lo más importante de la documentación es que sea de calidad. Parte de la documentación que se genera en proyectos informáticos, en ocasiones, carece de calidad. Suele verse con claridad en los manuales de usuario. Muchas veces, este tipo de manuales se encargan de describir obviedades de las ventanas o interfaces que cualquier usuario medio puede intuir delante del programa y omiten información importante que es de utilidad. La mayoría de las veces, esta falta de calidad viene dada porque el tiempo que se le dedica a la documentación es escaso. En ocasiones, el problema es que a los programadores no les gusta crear documentación y se hace de mala gana, luego eso se traduce en un pésimo resultado.

Para hacer una escritura de documentación de calidad, hay que seguir ciertas pautas:

1. Hacer siempre esquemas antes de ponerse a escribir. También pueden incorporarse dichos esquemas a los distintos manuales.
2. Clasificar la información según importancia. Es mejor tener un buen documento principal y muchos anexos que un documento principal muy grande.
3. Realizar resúmenes, esquemas, documentos maestros, etc. Siempre es bueno hacer un trabajo de síntesis en el que se explique el conjunto de la documentación.
4. Muchos lenguajes de programación tienen sus propios estándares y herramientas de documentación. Si es así, es mejor utilizarlos. Por ejemplo, Java tiene Javadoc.
5. A la hora de escribir la documentación para el usuario, hay que anticiparse y responder a las preguntas que pueda hacerse el usuario o lector. Habrá que responder a preguntas como: y ahora, después, ¿qué hago? ¿Qué hace este botón? ¿Qué implica borrar esto o aquello?, etc.

6. Es importante la claridad a la hora de escribir documentación de usuario. Hay empresas de software a las que les gusta utilizar segunda persona (tu) en vez de tercera persona (el usuario).
7. Hay que elegir bien la herramienta de documentación. En ciertos casos, obcecarse con herramientas como procesadores de textos hace difícil seguir el hilo a la documentación de distintos programas y módulos, quizá sería mejor utilizar herramientas hipertextuales o herramientas que facilitan la documentación del código como Help-Logix, Robo-Help. Doc-To-Help, etc. Tampoco hay que desechar herramientas colaborativas como wikis, Google docs y demás.
8. Muchas veces, se generan dos documentos de usuario para el software. La guía de usuario y el manual de referencia. Un manual de referencia explica el software de una forma más en profundidad. En ocasiones, este manual de referencia se distribuye como ficheros de ayuda dentro de la propia aplicación. La guía de usuario es más sencilla y explica cómo realizar paso a paso ciertas tareas con el software. Suelen tener formato de tutorial.
9. Hay que tener en cuenta el nivel del usuario que va a leer la documentación. En estos documentos, hay que dejar claro qué es lo que el usuario no debe ni puede hacer, las consecuencias de ejecutar algo, qué ocurre si no se hace algo...

Tipos de documentación

Fase Inicial.

En esta fase, se planifica el proyecto, se hacen estimaciones, se conviene si el proyecto es rentable o no, etc. Es decir, se establecen las bases de cómo van a desarrollarse el resto de fases del proyecto. En un símil con la construcción de un edificio, sería ver si se dispone de licencia de construcción, cuánto va a costar el edificio, cuántos trabajadores van a necesitarse para construirlo, quien va a construirlo, etc. La fase de planificación y estimación es la más compleja de un proyecto. Para esto se necesita gente con experiencia tanto en la elaboración de proyectos como en las plataformas en las que va a realizarse. De esta fase, surgen muchos documentos tanto de planificación (general y detallada) como documentos de estimaciones en los que se incluyen datos económicos, posibles soluciones al problema y sus costes, etc. Son documentos que se realizan en un alto nivel y se acuerdan con la dirección de la empresa o las personas responsables del proyecto. En estas fases iniciales, suelen tomarse decisiones que, a veces, afectan a todas las demás fases del proyecto, con lo cual tiene que estar todo bien documentado, detallado y soportado por datos concretos.

Tipos de documentación

Análisis.

En esta fase, se analiza el problema. Consiste en recopilar, examinar y formular los requisitos del cliente y analizar cualquier restricción que pueda aplicarse.

Todas las entrevistas con el cliente, por lo tanto, tienen que estar registradas en documentos y, generalmente, esos documentos se consensuan con el cliente, y algunos tienen hasta carácter contractual.

El jefe de desarrollo, en algunos proyectos, puede hacer firmar al cliente un documento de requisitos de la aplicación en el cual el equipo se compromete a realizar las especificaciones indicadas por el cliente y también el cliente se compromete a no variar sus necesidades hasta, por lo menos, terminar una primera release. Como puede verse, es un documento que obliga a ambas partes a cumplir con lo acordado y, en muchas ocasiones, establece un marco de relación entre cliente y desarrollador.

Diseño.

Esta fase consiste en determinar los requisitos generales de la arquitectura de la aplicación y dar una definición precisa de cada subconjunto de la aplicación. En esta fase, los documentos ya son más técnicos. Suelen crearse dos documentos de diseño:

1. uno más genérico, en el que se tiene una visión de la aplicación más general
2. otro detallado, en el que se profundizará en los detalles técnicos de cada módulo concreto del sistema.

Estos documentos los realizarán los analistas y/o analista programadores.

Codificación o implementación.

Esta fase consiste en la implementación del software en un lenguaje de programación para crear las funciones definidas durante la etapa de diseño. Durante esta fase, se crea documentación muy detallada en la que se incluye código. Aunque mucho código suele comentarse en el mismo programa, también tienen que generarse documentos donde se indica, por ejemplo, para cada función, las entradas, salidas, parámetros, propósito, módulos o librerías donde se encuentra, quién la ha realizado, cuándo, las revisiones que se han realizado, etc. Como puede observarse, el detalle es máximo teniendo en cuenta que ese código, en un futuro, va a tener que ser mantenido por la misma o, seguramente, otra persona y toda información que pueda recibir, a veces, es poca.

Pruebas.

En esta fase, se realizarán pruebas para garantizar que la aplicación se programó de acuerdo con las especificaciones originales y los distintos programas de los que consta la aplicación están perfectamente integrados y preparados para la explotación. Como se ha visto anteriormente, las pruebas son de todo tipo. Puede clasificarse la documentación de las pruebas en dos bloques diferentes.

1. Existen unas pruebas funcionales en las que se prueba que la aplicación realiza lo que tiene que hacer con las funciones acordes a los documentos de especificaciones que se han establecido con el cliente, y esas pruebas deberían llevarse a cabo con el cliente delante. Mientras están realizándose las pruebas, tiene que hacerse todo tipo de anotaciones para luego plasmarlas en un documento, al que tendrá que dar su visto bueno el cliente (que, por ese motivo, estuvo en las pruebas). En ese documento, van detallándose fallos tanto de la propia aplicación como de modificaciones realizadas si no cumplen con las especificaciones iniciales. En el caso de que haya discrepancia, suele consultarse documentación anterior para que no se incluyan en este punto funcionalidades nuevas o variaciones de las funcionalidades.

Tipos de documentación

2. En otro documento aparte, se detallaran los resultados de las pruebas técnicas realizadas. A estas pruebas, ya no hace falta que asista el usuario o cliente, puesto que son de carácter meramente técnico. En estas pruebas, se harán cargas reales, se someterá la aplicación y el sistema a estrés, etc.

Explotación

En esta fase, se instala el software en el entorno real de uso y se trabaja con él de forma cotidiana. Generalmente, es la fase más larga y suelen surgir multitud de incidencias, nuevas necesidades, etc. Toda esta información suele detallarse en un documento en el que se registran los errores o fallos detectados intentando ser lo más explícito posible, puesto que luego los programadores y analistas deben revisar estos fallos o bugs y darles la mejor solución posible. También surgirán otras necesidades que deben ir detallándose en un documento y que pasaran a realizarse en operaciones de mantenimiento.

Mantenimiento

En esta fase, se realiza todo tipo de procedimientos correctivos (corrección de fallos) y actualizaciones secundarias del software (mantenimiento continuo) que consistirán en adaptar y evolucionar las aplicaciones. Para realizar las labores de mantenimiento, hay que tener siempre delante la documentación técnica de la aplicación. Sin una buena documentación de la aplicación, las labores de mantenimiento son muy difíciles y su garantía en ese caso sería escasa. Todas las operaciones de mantenimiento tienen que estar documentadas porque tiene que conocerse quién ha realizado la operación, qué ha hecho y cómo. También tienen que estar documentadas porque deberían probarlas otra persona distinta al programador.



Es importante tener el código comentado y la mejor de las opciones es comentar el código dentro de él. Otras opciones implican mantener documentos que la mayoría de las veces se quedan obsoletos y desactualizados. En Java, el código se documenta en el mismo fichero y, con una herramienta llamada Javadoc, pueden extraerse los textos y comentarios de dicho código fuente y transformarlos en páginas web con formato HTML. Los comentarios, como saben los programadores de Java, se insertan entre los caracteres `/**` y `*/`. Además del comentario, pueden incluirse otras etiquetas más específicas como las siguientes:

- `@author`. Como su nombre indica, identifica al autor del código fuente.
- `@version`. Esta etiqueta identifica la versión del código.
- `@return`. Especifica el valor de retorno de la función o método. No hay que especificar dicha etiqueta si la función devuelve un valor void.
- `@since`. Esta etiqueta especifica la fecha de creación del programa.
- `@deprecated`. Especifica que esta función, método o la misma clase está obsoleta.
- `@param<nombre_par>`. Describe el parámetro "nombre_par". Suele especificarse cada parámetro en una línea.
- `@see`. Se utiliza esta etiqueta si quiere redirigirse al que lee el código a otro apartado.

- `@link<URL>`. Se utiliza esta etiqueta si quiere dirigirse al usuario a una URL en concreto. Véase un ejemplo de documentación muy sencillo:

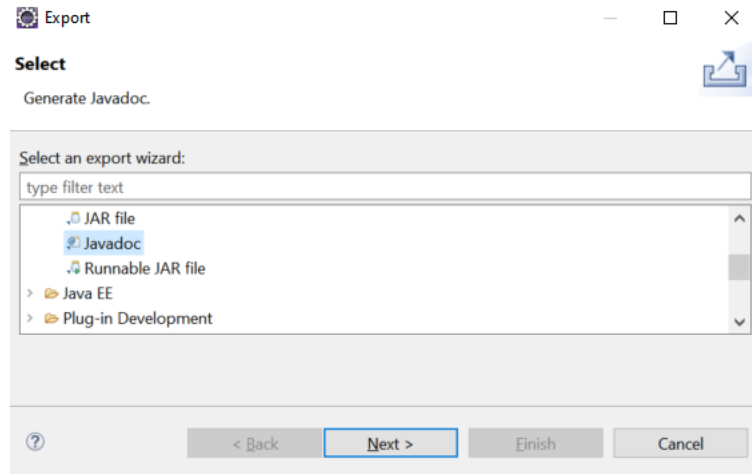
```
package test;

/**
 * @version 1.0
 * @author manuel
 * @since 06/06/2020
 */
public class Main {

    /**
     * Metodo encargado de arrancar el proyecto
     *
     * @param args arguments
     */
    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}
```

Javadoc

Para generar la documentación se puede hacer con el propio IDE como Eclipse o NetBeans o desde línea de comandos con `> javadoc main.java`. Para hacerlo con Eclipse podemos hacer botón derecho sobre el proyecto y seleccionar `export > javadoc`



Actividad 9

Crea un proyecto y crea un javadoc.